v1.0.0
20260313

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.5]
VISUAL FORMALISMS

VISUAL FORMALISMS

Design, Model Checking e Verifica (affrontato nelle vecchie slide)
Rappresentare dinamica di un sistema concorrente tramite Visual Formalism

- Formalisms for rigorously describing models of concurrent systems by means of some kind of visual diagrams
  - representing system *behaviours / dynamics*
  - representing task structure and task dependencies
- Useful for both requirement specification, analysis and design
  - formal analysis when formally specified
- Formalisms considered in this module
  - **Petri Nets**
  - **Statecharts**
  - **Activity Diagrams**

Tre aspetti fondamentali di Ingegneria del Software da considerare nei Sistemi:
- Aspetto della struttura (per rappresentare struttura parte statica del sistema, uso Class Diagram; per rappresentare struttura parte dinamica del sistema (runtime), uso Component and Connector Diagram)
- Comportamento (uso State Diagram per rappresentare i vari stati del sistema; posso usare anche Rete di Petri)
- Interazioni (uso Sequence Diagram per rappresentare interazioni nel sistema, in modo sincrono)

Sono tre assi ortogonali da considerare (le varie rappresentazioni delle tre sono importanti)

PETRI NETS

LTS rappresenta il comportamento del sistema (visione globale del sistema)

PETRI NETS

Formalismo, molto generale, per rappresentare dinamica dei sistemi

Noi lo useremo in questo modo: dato un sistema a processi, come lo rappresento con una rete di petri, catturando gli elementi essenziali?

- Abstract, formal model of information flow
  - describing and analysing the **flow of information** and **control** in systems
  - particularly systems that may exhibits asynchronous and concurrent activities
- Introduced by Carl Adam Petri ~ 1965
  - further developed, extended and adopted in many computer science contexts
- Major use
  - modelling of systems of events *in which it is possible for some events to occur concurrently* but there are *constraints* on the concurrence, precedence, or frequency of these occurrences

La differenza principale tra PN e LTS sta nel fatto che le Reti di Petri sono un modello basato sullo stato locale e sui processi, mentre gli LTS sono un modello basato sullo stato globale e sulle interazioni.

1. Un LTS è un grafo orientato che rappresenta il comportamento di un sistema attraverso stati e transizioni. Visione Globale: Ogni nodo del grafo rappresenta l'intero stato del sistema in un dato momento. Interazione: Le frecce (transizioni) sono etichettate con azioni (a, b, \tau). Ci dicono cosa accade per passare da uno stato all'altro. Esplosione degli stati: Poiché ogni combinazione di variabili o processi genera un nuovo stato globale, il numero di nodi può diventare enorme (problema della state-space explosion).
2. Le Reti di Petri sono grafi bipartiti composti da piazze, transizioni e token.Visione Locale: Non esiste un unico nodo per lo stato globale. Lo stato è dato dalla distribuzione dei token nei vari posti (chiamata marcatura). Concorrenza Esplicita: Le Reti di Petri mostrano chiaramente quali parti del sistema sono indipendenti. Due transizioni possono scattare contemporaneamente se hanno risorse (token) sufficienti, senza dover definire ogni possibile combinazione negli stati globali. Modellazione delle Risorse: Sono perfette per descrivere produttori/consumatori, mutua esclusione e sincronizzazione di risorse condivise.

Le reti di petri sono molto efficaci perchè catturano il non-determinismo intrinseco che c'è nel sistema (e lo fanno in modo molto modulare)

Con PN, si può fare Model Checking (esplorare tutti i possibili scenari): in ogni stato di marcatura con un token, vado a vedere tutte le possibili evoluzioni facendo scattare tutte le transizioni che possono scattare

PETRI NET GRAPH

Le piazze rappresentano sia il concetto di stato globale (tramite le marcature) sia il concetto di stato locale (dove si trova il processo)
Le transizioni rappresentano il concetto di azione

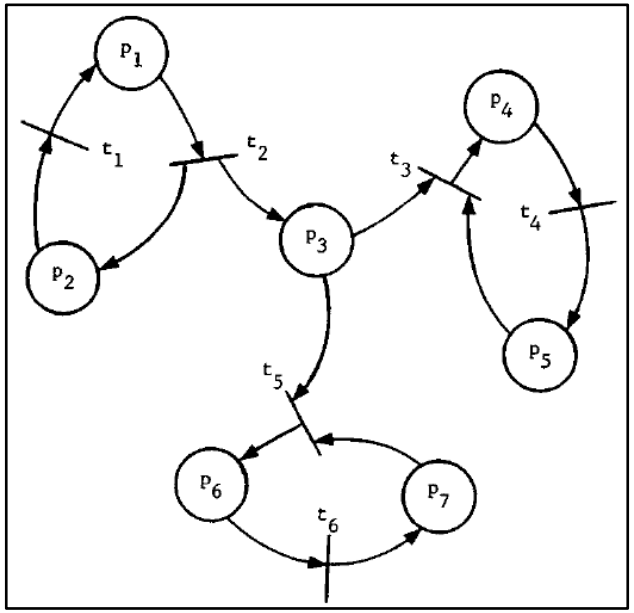
- Bi-partite graphs representing a Petri Net
  - two types of *nodes*
    - **places** (the circles) and **transitions** (the bars)
  - connected by directed arcs
    - from node i to node j: i is an input to j and i is an output of i

Non-determinismo: più transizioni possono scattare, se si hanno la quantità dei token in input

IMPORTANTE (all'esame): Se devi rappresentare un sistema concorrente fatto da più processi che interagiscono, prima di tutto rappresenta i processi, solo dopo vai a vedere le interazioni che ci sono

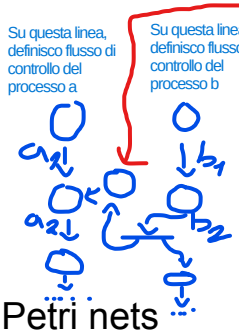
Le reti di Petri non danno solo la rappresentazione della struttura, ma, grazie ai token, posso fare una fotografia "a runtime" dello stato in cui si trova il sistema e quali sono le possibili evoluzioni di quello stato. Il token, messo in una piazza, mi dice che lo stato di quel processo si trova in quel punto: il token rappresenta un flusso di controllo, un istante di quel processo (processo rappresentato come PN), che segue uno schema di comportamento che è definito dal programma che abbiamo detto.

Se uso due token in un PN, significa che ho due processi che hanno quella struttura



Le ottimizzazioni della PN bisogna farle dopo aver capito bene ed aver definito il PN finale

Per risolvere il problema della sincronizzazione tra azioni (example: a2 da eseguire dopo b2) tra processi, uso un semaforo ad eventi (init. a 0): per modellarlo con PN, usiamo token e piazze per modellare condizioni/situazioni di stato (cattura il senso del semaforo: uso una piazza che quando ho un token in quella piazza, significa che a2 è stata eseguita prima di eseguire b2 che prima di eseguire b2 devo avere un token sulla piazza che definisce la condizione (messo dall'altro processo b) e sulla piazza del flusso di controllo del processo a)



Piazza della condizione " b2 eseguita" che mi pemette di eseguire a2 dopo b2 (se a1 eseguita)

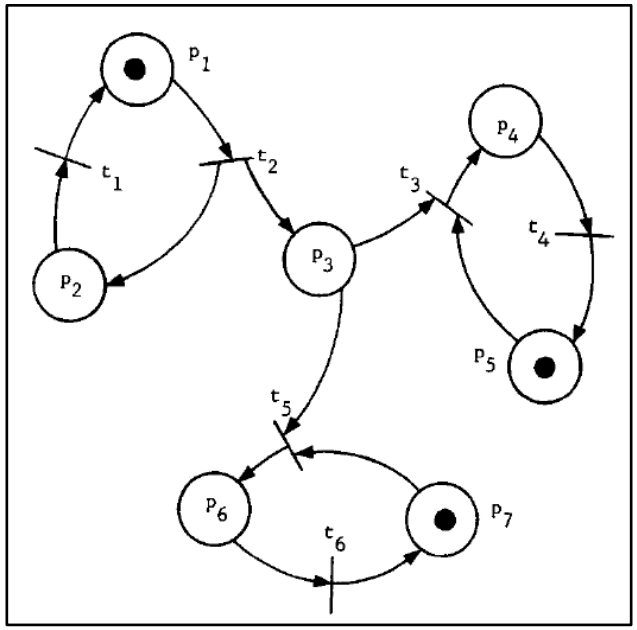
PN di a e b (problema di sincronizzazione)

- Models the *static* properties of the system

TOKENS AND MARKINGS

Le Reti di Petri con archi inibitori sono un'estensione delle reti di Petri classiche utilizzate per modellare sistemi distribuiti discreti in cui è necessario verificare l'assenza di token in un posto.

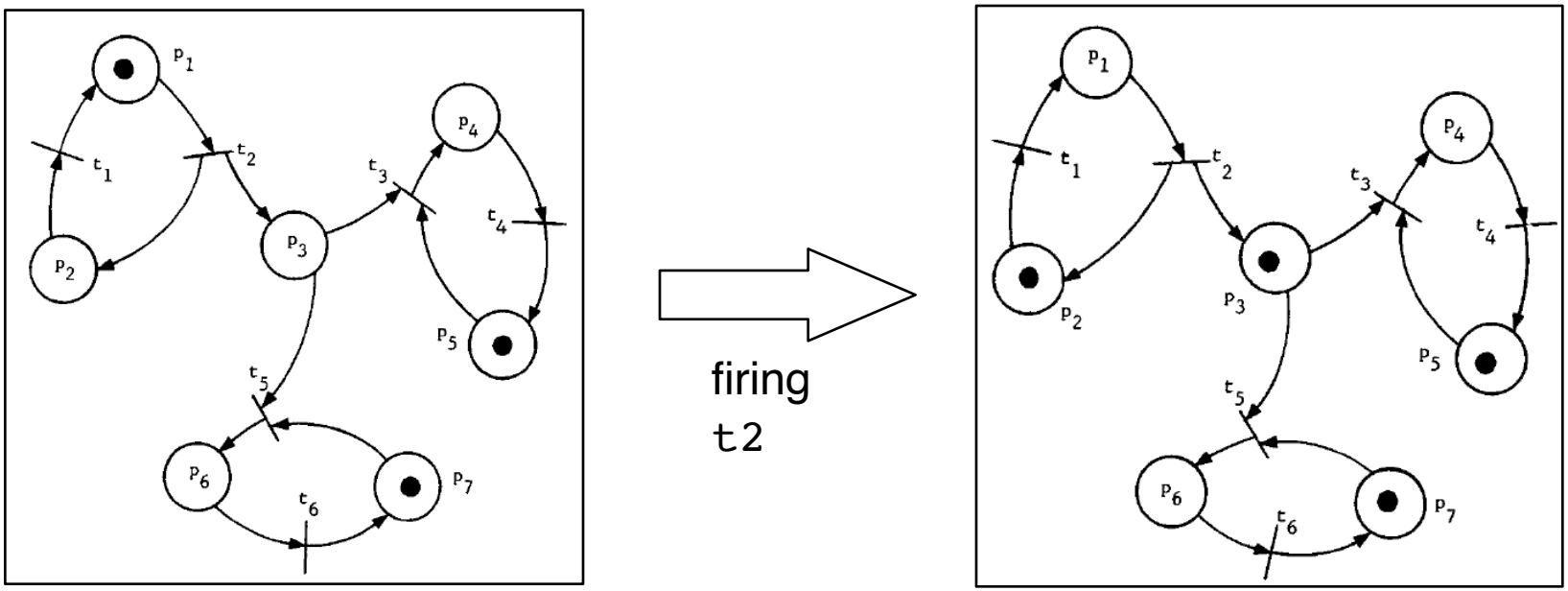
- In addition to the static properties represented by the graph, a Petri Net has *dynamic properties* that result from its *execution*
  - the execution of a Petri net is controlled by the position and movement of markers called **tokens** in the net



  - tokens are indicated by black dots, residing in places
- A Petri Net with tokens is called **marked Petri Net**

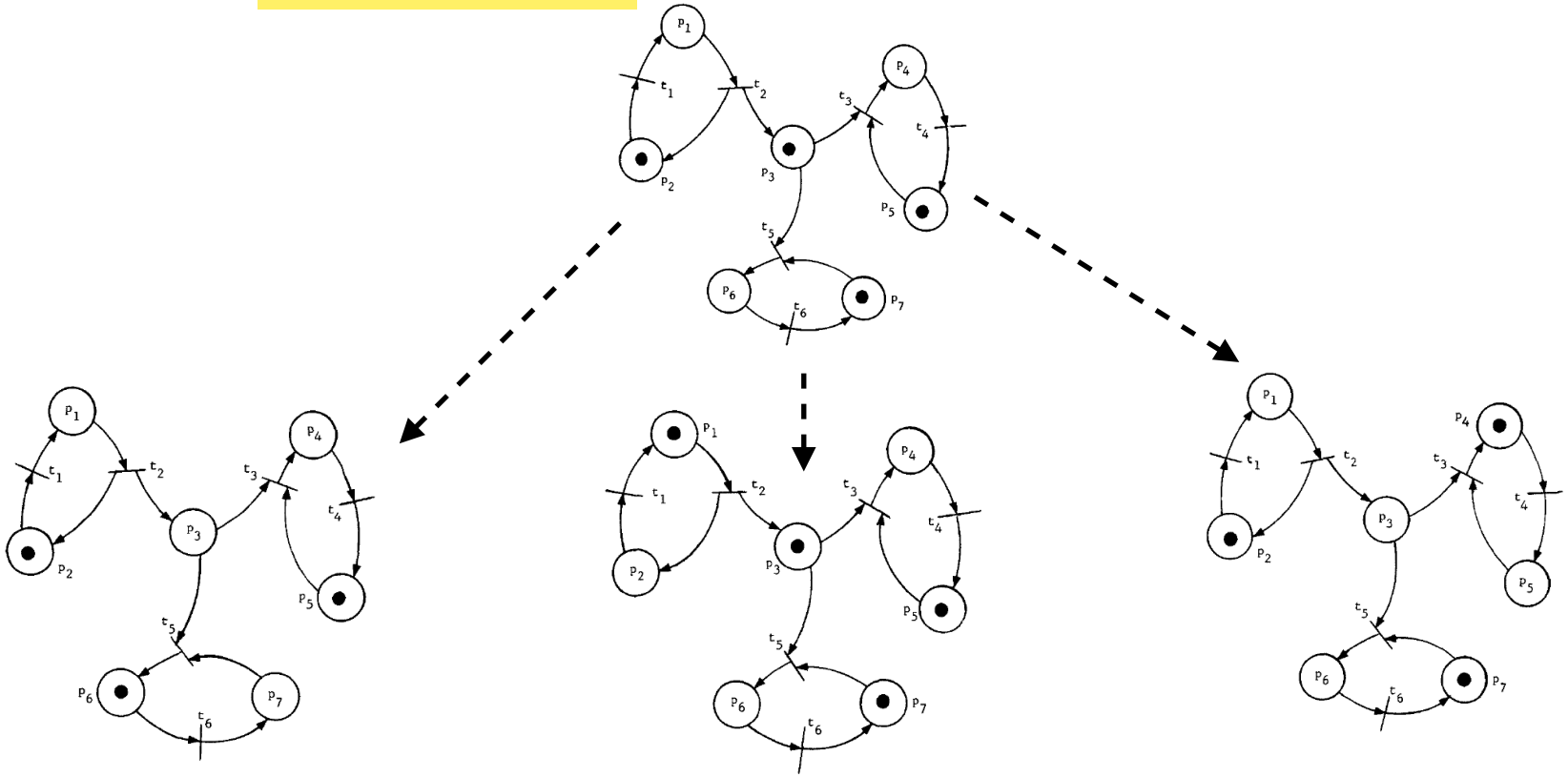
EXECUTION RULES

- Tokens are moved by the *firing* of transitions of the net
  - a transition must be *enabled* in order to fire
    - a transition is enabled when all of its input places have a token in them
  - the transition fires by removing the enabling tokens from their input places and generating new tokens which are deposited in the output place of the transition



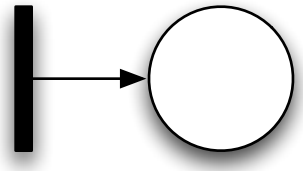
MARKINGS

- The distribution of tokens in a marked Petri Net defines the state of the net and is called **marking**
  - the marking may change as a result of firing transitions
- Different transitions may fire, with different result markings
  - inherent *non-determinism*

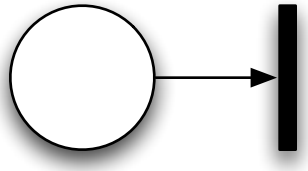


TRANSITIONS ON THE BOUNDARY

- *source* transition
  - without any input place
  - just produce tokens



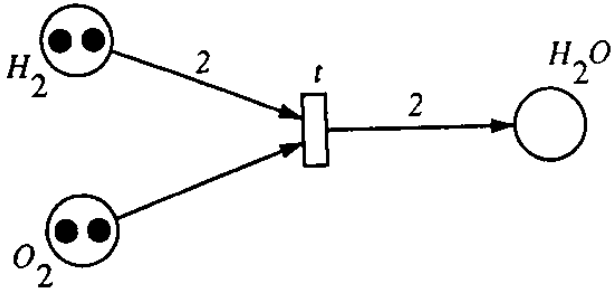
- *sink* transition
  - without any output place
  - just consume tokens

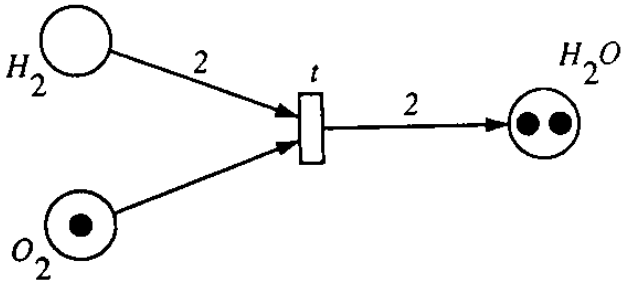


WEIGHTED ARCS

- A variant consider also weights for arcs:
  - a transition `t` is enabled if each input place `p` of `t` is marked with at least `w(p,t)` tokens, where `w(p,t)` is the weight of the arc from `p` to `t`
  - a firing of an enabled transition `t` removes `w(p,t)` tokens from each input place `p` of `t`, and adds w_1 tokens to each output place `p` of `t`, where `w(p,t)` is the weight of the arc from `t` to `p`

- Reaction example: $2H_2 + O_2 \rightarrow 2H_2O$

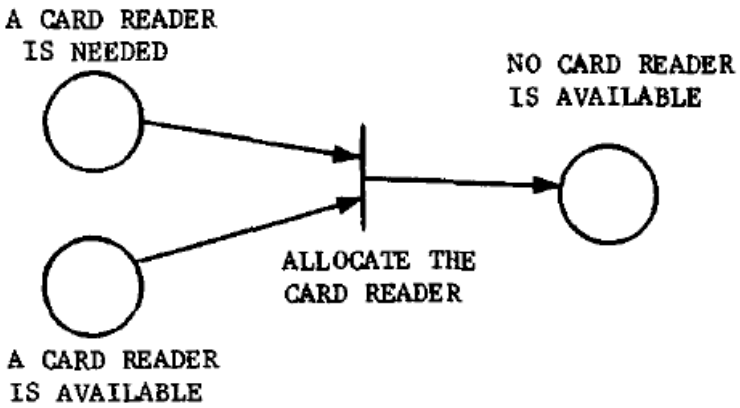


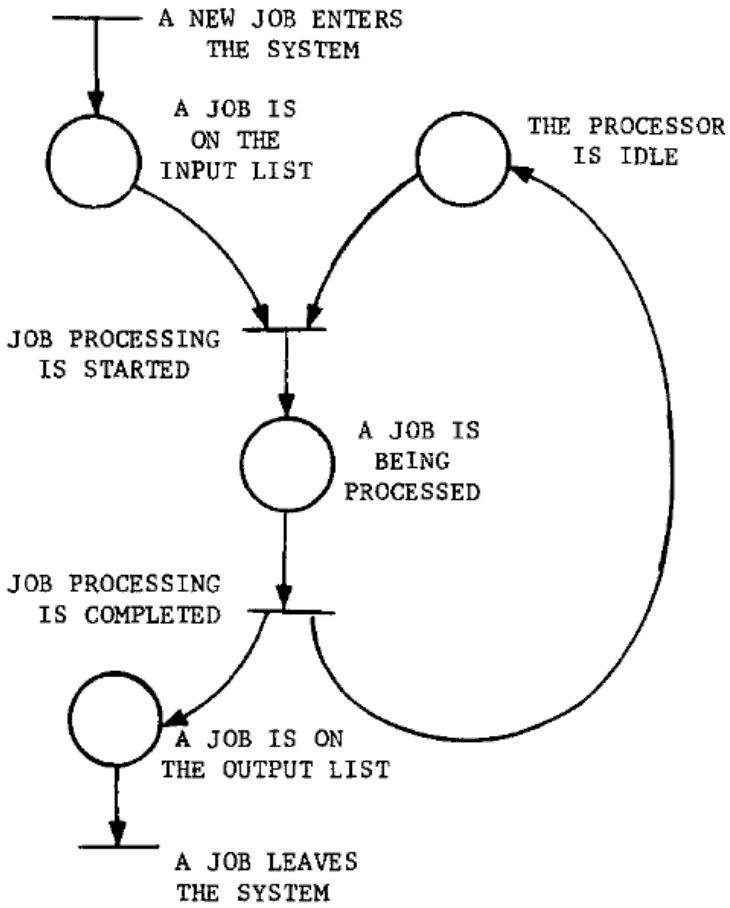


MODELLING WITH PETRI NETS

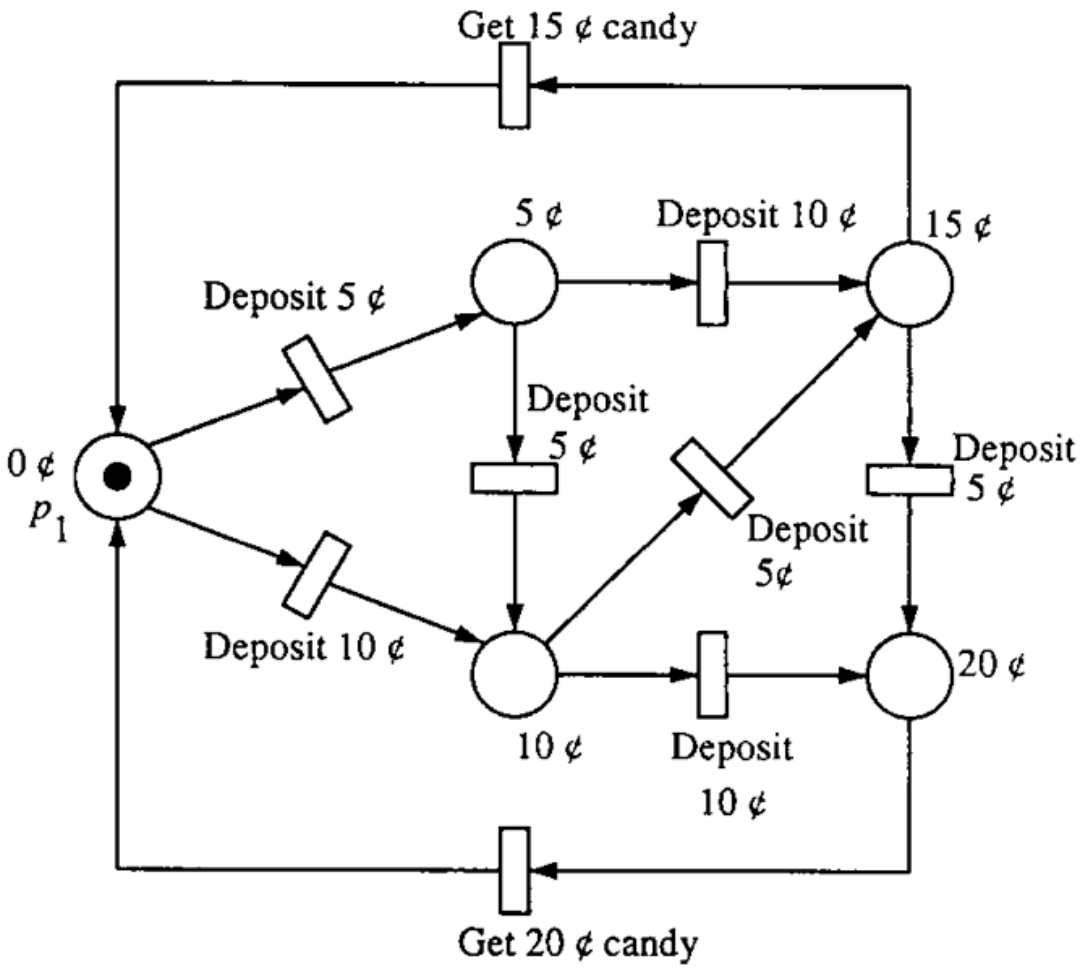
- Petri nets can be used to model quite naturally concurrent systems in terms of:
  - **events** and **conditions**
  - the relationships among them
- Interpretation
  - in a system at any given time certain conditions will hold
  - the fact that these conditions hold may cause the occurrence of certain events
  - the occurrence of events may change the state of the system
    - causing some of the previous conditions to cease holding and causing other conditions to begin to hold
  - firing of a transition = occurrence of an event
    - considered instantaneous or better: *atomic change* of the system

EXAMPLE: RESOURCE ALLOCATION





EXAMPLE: A VENDING MACHINE



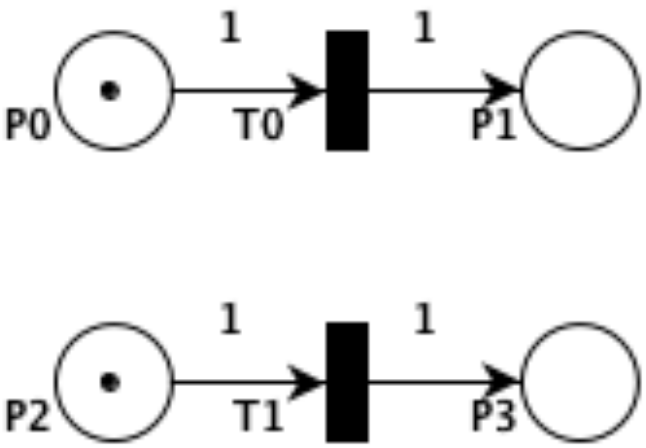
INTERPRETATIONS

- Typical interpretations of places and transitions

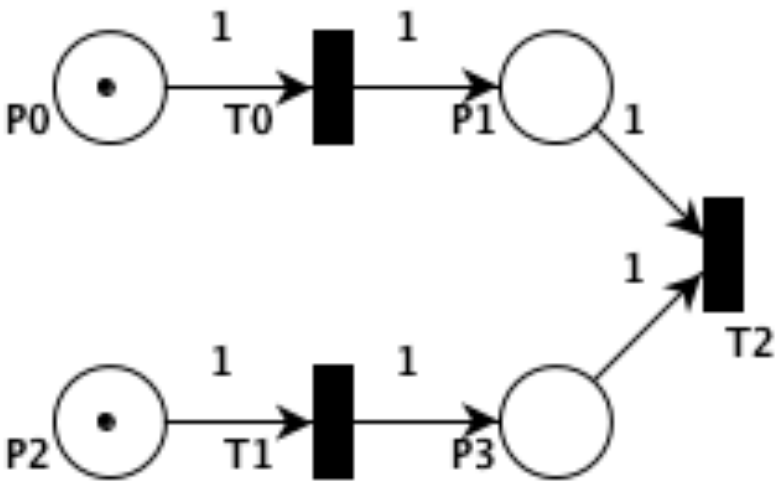
Input places	Transition	Output places
Pre-conditions	Event	Post-conditions
Input data	Computational step	output data
Input signals	Signal processor	Output signals
Resource needed	Task or Job	Resource Released
Conditions	Clause in Logic	Conclusion(s)
Buffers	Processor	Buffers

MODELLING CONCURRENCY AND PARALLELISM

- PN are ideal for modelling systems of distributed control with multiple processes occurring concurrently

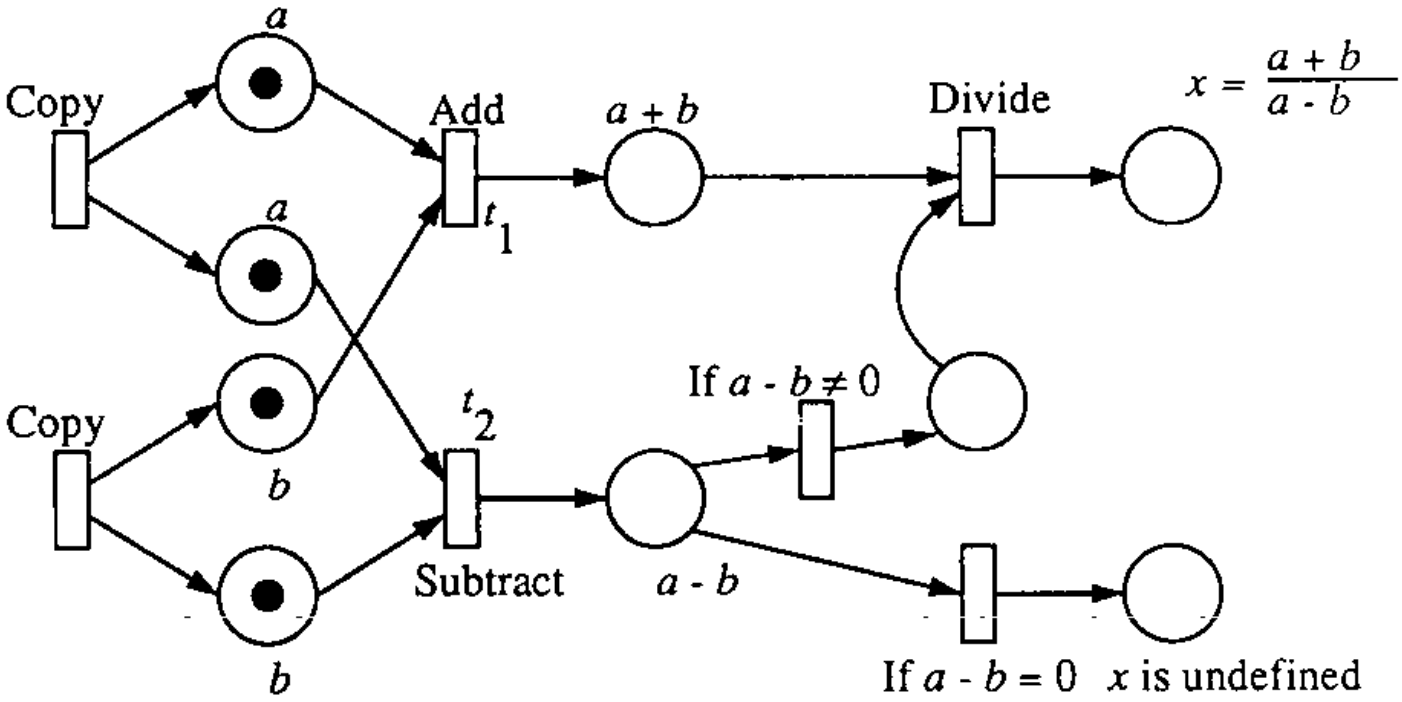


Modelling two parallel activities



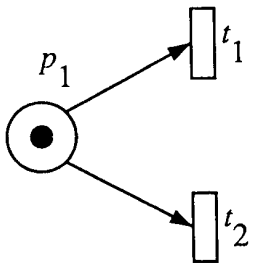
Modelling two parallel activities with synchronisation

DATA-FLOW COMPUTATION



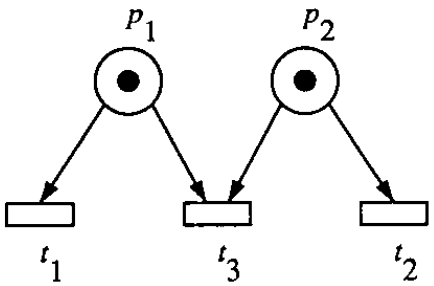
MODELLING CONFLICT AND CONCURRENT EVENTS

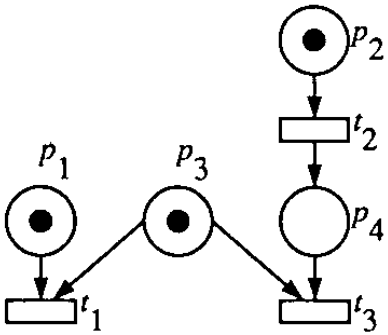
- Representing conflicting or choice events:



Si ha un conflitto quando due transizioni "competono" per la stessa risorsa (lo stesso gettone). L'attivazione di una transizione disabilita l'altra.Condizione: Due transizioni e1 ed e2 sono in conflitto se condividono almeno un posto in ingresso e quel posto non ha abbastanza gettoni per soddisfare entrambe contemporaneamente. Comportamento: Il sistema deve compiere una scelta non deterministica. Se scatta e1, il gettone sparisce ed e2 non può più scattare.

- **Conflict events** vs. **concurrent events**
  - two events e1 and e2 are *in conflict* if either e1 or e2 can occur but not both
  - two events e1 and e2 are *concurrent* if both events can occur in any order without conflicts
- A situation where conflict and concurrency are mixed is called a *confusion*





Due eventi sono concorrenti quando sono indipendenti l'uno dall'altro. Possono avvenire nello stesso momento o in qualsiasi ordine senza influenzarsi. Condizione: Le transizioni e1 ed e2 sono abilitate e i loro insiemi di posti in ingresso sono disgiunti (o il posto comune ha gettoni sufficienti per entrambe). Comportamento: L'ordine non conta. Se il sistema esegue prima e1 e poi e2, il risultato finale sarà identico a quello ottenuto eseguendo prima e2 e poi e1.

ASYNCHRONY AND LOCALITY

Il tempo è "logico", non cronometrico. Ciò che conta è cosa deve accadere prima di cos'altro.

- In a PN there is no inherent measure of time or the flow of time
  - the only important property of time, from a logical point of view, is in defining a **partial ordering** of the occurrence of events
  - events which need not be constrained in terms of their relative order of occurrence are not constrained
- **Locality**
  - in a complex systems composed by independent asynchronously operating subparts each part can be modelled by a Petri Net
  - the enabling and firing of transitions are then affected by, and in turn affect only, *local changes* in the marking of the Petri Net

La località è ciò che rende le Reti di Petri perfette per modellare sistemi distribuiti.
- Effetto Locale: Per sapere se una transizione T può scattare, non devi guardare l'intera rete. Devi guardare solo i suoi posti in ingresso (il suo preset).
- Cambiamento Locale: Quando la transizione scatta, modifica solo i suoi posti in ingresso e i suoi posti in uscita. Il resto della rete (lo stato dei posti lontani) rimane invariato e non viene "interpellato".

NON-DETERMINISM

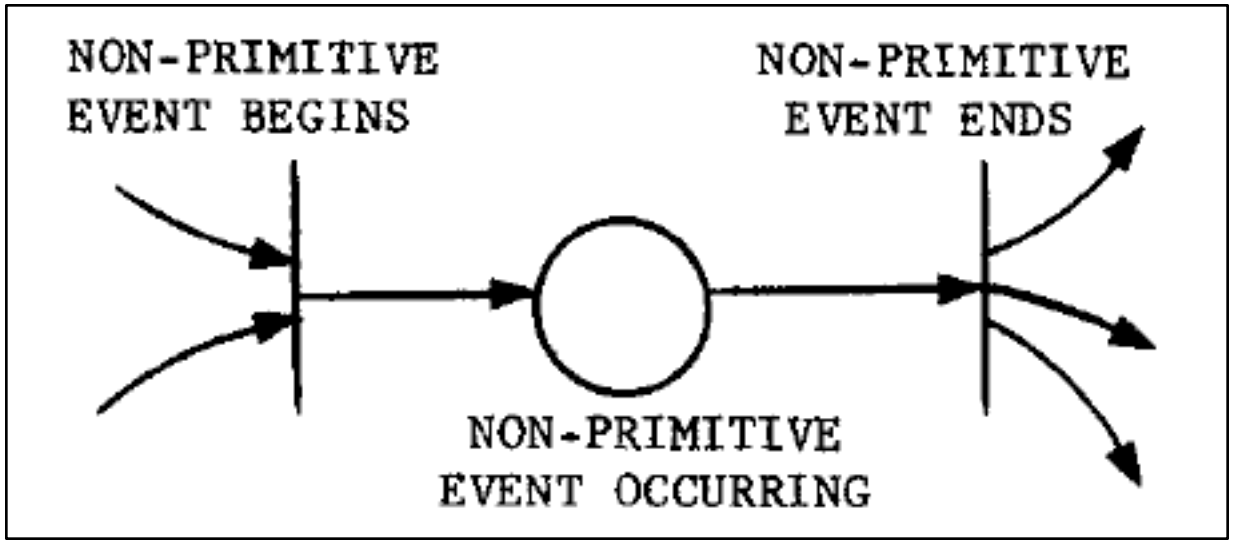
- Naturally modelling *non deterministic* behaviours
  - a PN is viewed as a sequence of discrete events whose order of occurrence is one of the possibly many allowed by the basic structure
  - if at any time more than one transition is enabled, then any of the several enabled transitions may fire
  - the choice as to which transition fires is made in a nondeterministic manner
    - randomly or by forces that are not modelled

ATOMIC VS. NON-ATOMIC EVENTS

- The occurrence of (primitive) events is instantaneous
  - non primitive events (with a duration) must be modelled by multiple events
  - e.g.: activity with a beginning and an ending event

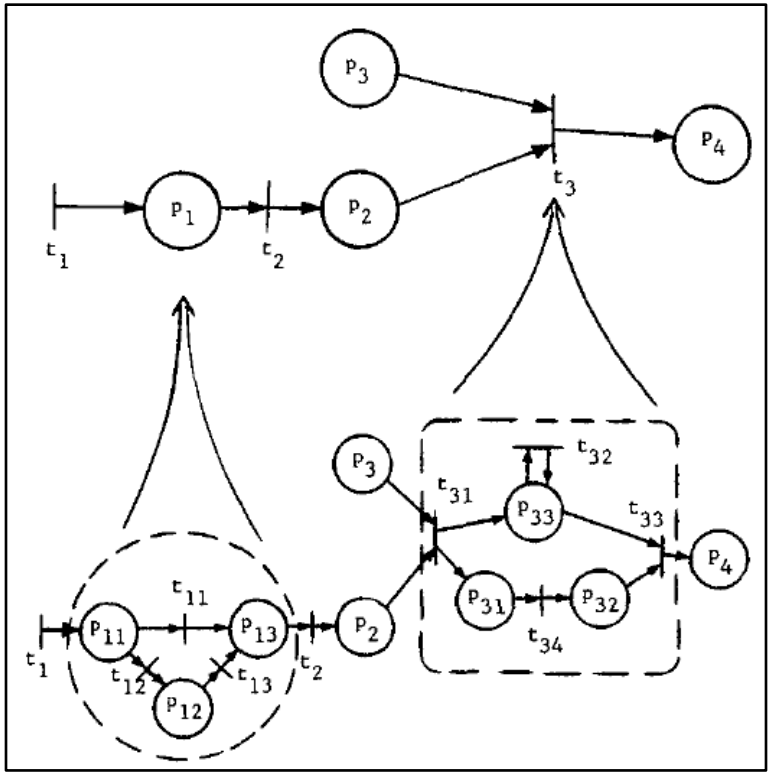
Un evento atomico è istantaneo. Accade o non accade, senza stati intermedi.

Per modellare un'attività con una durata, dobbiamo "spacchettarla" usando la struttura della rete di Petri. Il pattern standard prevede:
- Transizione di Inizio: Rappresenta l'istante in cui l'attività comincia.
- Piazza di "Esecuzione": Rappresenta lo stato in cui l'attività è in corso (il tempo scorre qui).
- Transizione di Fine: Rappresenta l'istante in cui l'attività termina.



HIERARCHIES

- Natural support to model hierarchies
  - an entire net may be replaced by a single place or transition for modelling at a more abstract level (**abstraction**)
  - places and transitions may be replaced by subnets to provide more detailed modelling (**refinement**)



APPLICATION TO CONCURRENT DESIGN AND PROGRAMMING

- PN can be naturally used to model software systems, concurrent software systems in particular

Le Reti di Petri (PN) non sono solo astrazioni matematiche; sono uno strumento potentissimo per il design del software perché permettono di visualizzare ed evitare bug logici (come i deadlock) prima ancora di scrivere una sola riga di codice.

A
- Representing problems
  - critical sections and mutual exclusion problems
  - synchronisation

B
- Representing mechanisms behaviour
  - semaphores
  - synchronisers

C
- Representing entire problems
  - Producers / Consumers
  - Readers-Writers
  - Dining Philosophers

Le Reti di Petri sono famose per saper risolvere e visualizzare i problemi classici:

In pratica, disegnare la Rete di Petri di un algoritmo concorrente è come fare un test di stress logico prima di implementarlo tramite codice.

A

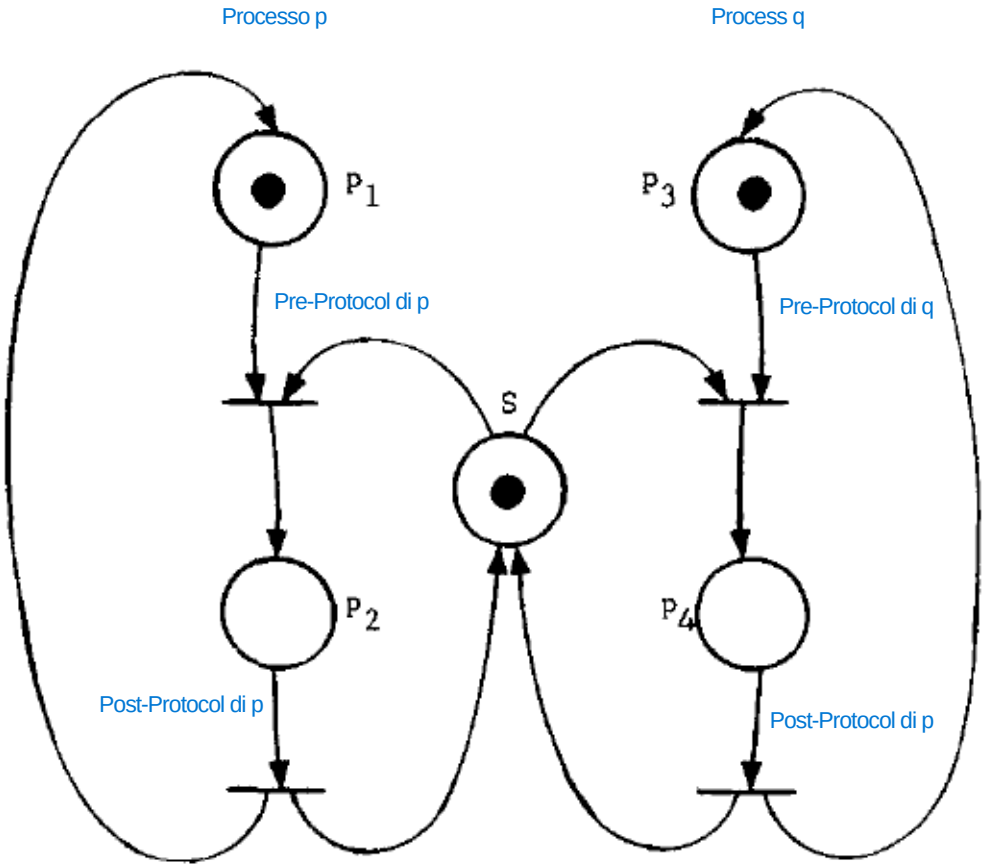
CRITICAL SECTION AND MUTUAL EXCLUSION PROBLEMS

- p2 and p4 represent critical sections (p1 e p3 rappresentano Non Critical Section)
  - s is the token needed for entering in CS (è la condizione/permesso di entrare nella CS: un permesso di entrata, cioè la modellazione del semaforo inizializzato con 1, cioè numero di permessi)

La rete di Petri deve rimanere Bounded (numero di token preservati)

Siamo noi che definiamo il livello di astrazione che vogliamo (come in tutti i formalismi)

Come modello la generazione di Thread? Dato un token, un transizione ne genera due (da un flusso di controllo a due)



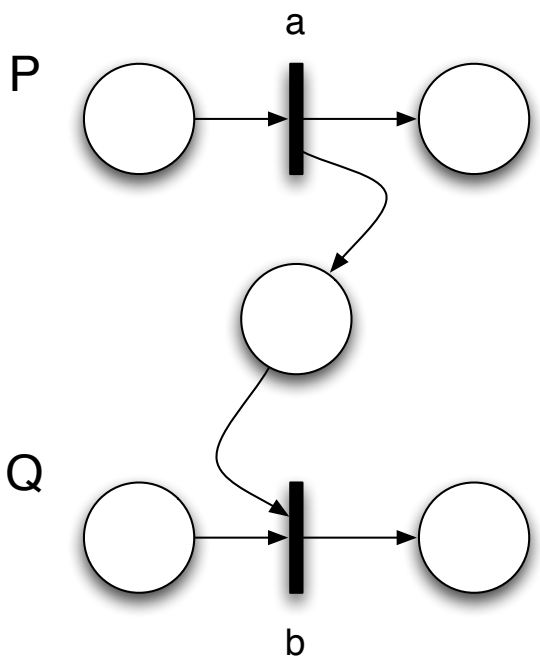
Usando la piazza s, solo un processo può entrare in sezione critica (perché competono sul token)

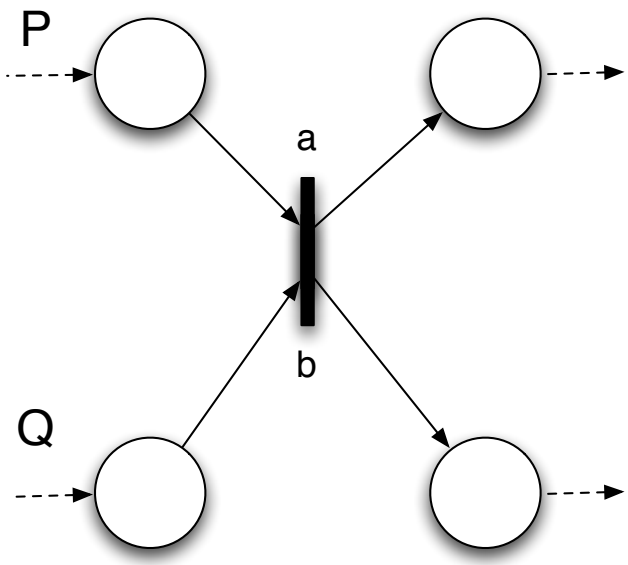
Un token significa un istanza di un processo (il suo flusso di controllo) e le piazze rappresentano lo stato del processo. Ma il token non rappresenta solo quello, ma anche la quantità di permessi, in questo caso. Quindi, con un solo formalismo (PN con token e piazze) catturo tutti gli aspetti che vogliamo modellare

Si poteva avere un unica struttura anzichè due (ma sarebbe stato più difficile capirlo in fase di progettazione, si può ottimizzare dopo). Ho fatto due strutture identiche perchè voglio definire qual'è l'invariante/vincolo che voglio avere, che con la PN deve descrivere il fatto che la sezione critica vale, cioè vale la mutua esclusione (cioè non devo avere un Marking della PN in cui ho un token in p2 ed uno in p4)

A SYNCHRONIZATION

- Imposing an order between process actions
  - process actions represented by transitions
  - relating actions (transitions) through conditions (places)



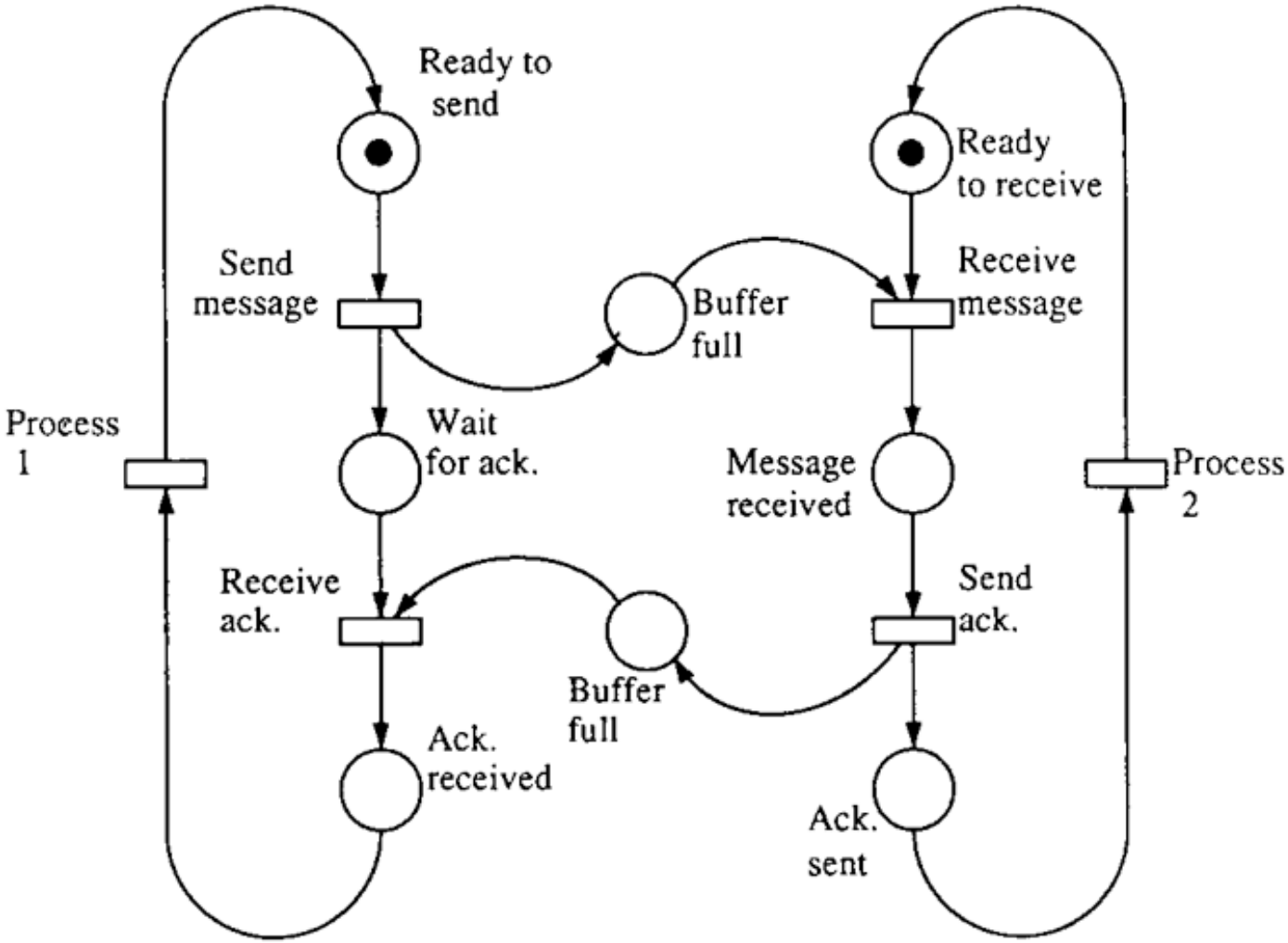


- b action of process Q can be executed after the execution of action a of process P

- b action of process Q and a action of process P must be executed synchronously

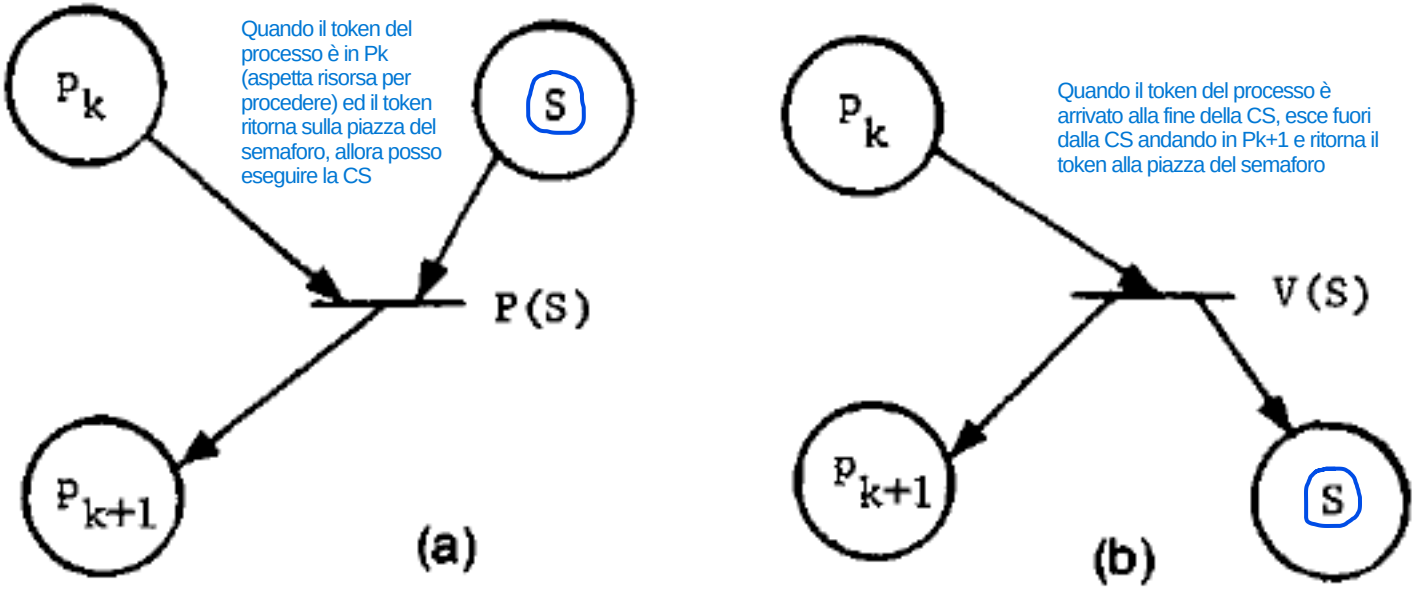
A COMMUNICATION

Bounded Buffer relativo al Messaging



B SEMAPHORES

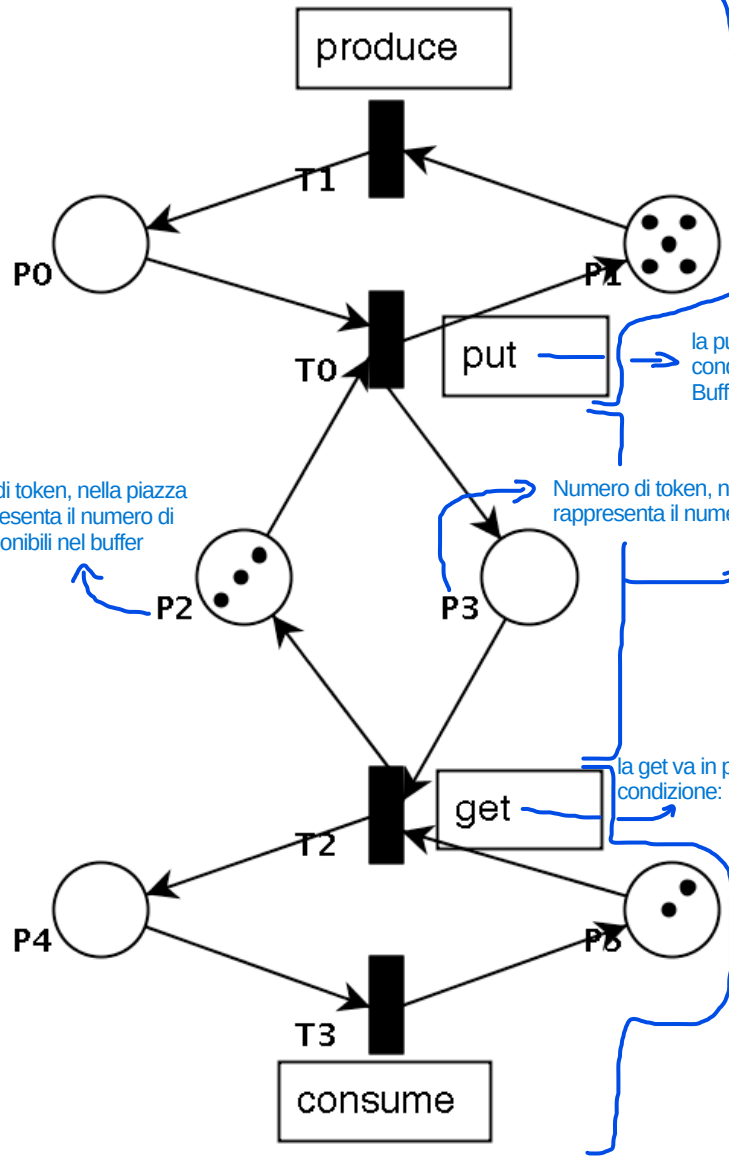
- Semaphore modelled as a shared resource (place)
  - a– modelling wait (P) as a transition with the semaphore res. as input place
  - b– modelling signal (Q) as a transition with the semaphore res. as output place



PRODUCERS AND CONSUMERS

Ho N produttori, quindi N token

Quando voglio rappresentare una cosa come Atomica, la definisco come Transizione (non voglio avere interleaving)



PRODUCERS

la put va in porto se ho producer che produce item nel Buffer (piazza di condizione: P0). Se Buffer è Bounded, devo aggiungere un altra condizione: il Buffer ha ancora spazio dove mettere gli item prodotti (piazza di condizione: P2)

Numero di token, nella piazza P2, rappresenta il numero di posti disponibili nel buffer

Numero di token, nella piazza P3, rappresenta il numero di item nel Buffer

BOUNDED-BUFFER

La somma tra P2 e P3 deve essere costante (invariante)

la get va in porto se ho item nel Buffer (piazza di condizione: P3). Rimane uguale se Buffer Bounded o no

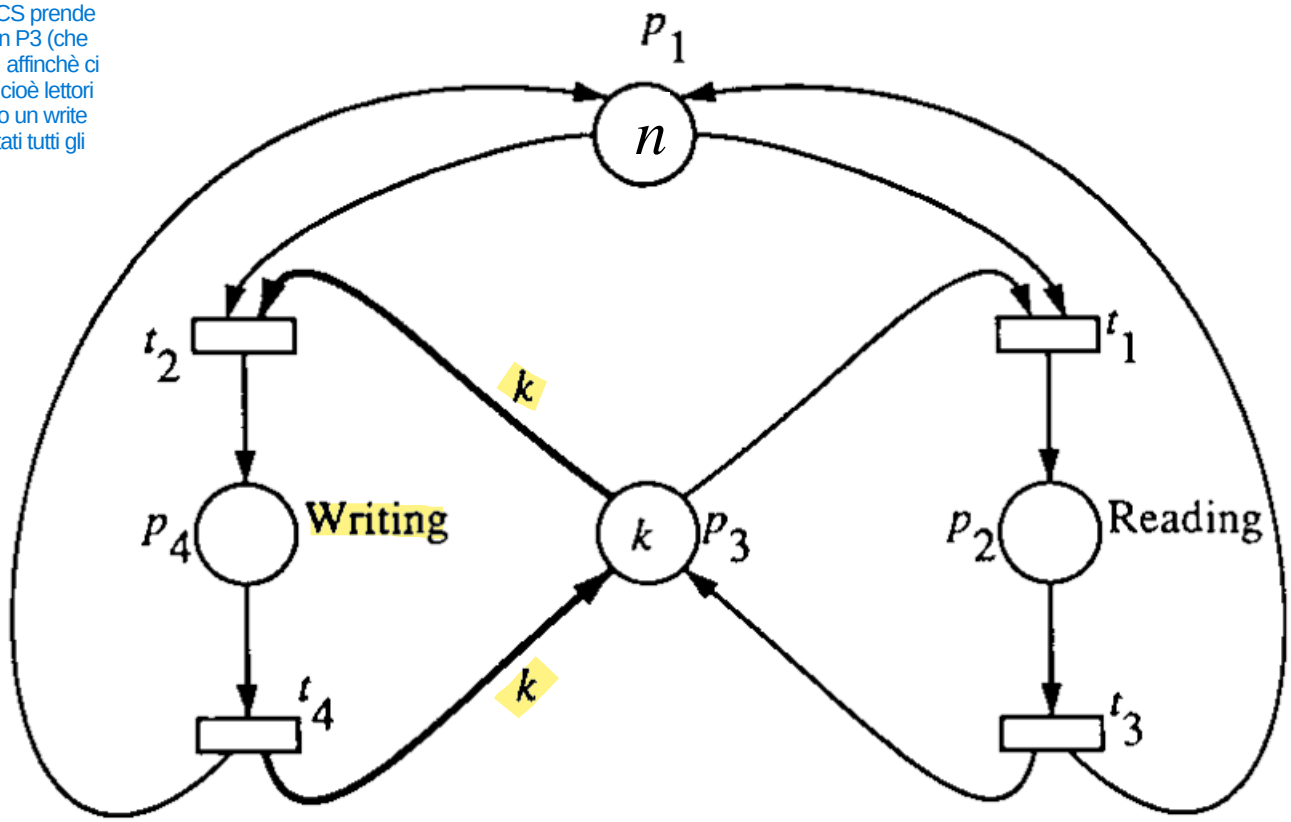
CONSUMERS

READERS-WRITERS

Invarianti:
1) Se c'è un lettore, non ci devono essere scrittori in CS
2) Ci possono essere più lettori in CS
3) Se c'è uno scrittore, non ci possono essere lettori ed altri scrittori in CS

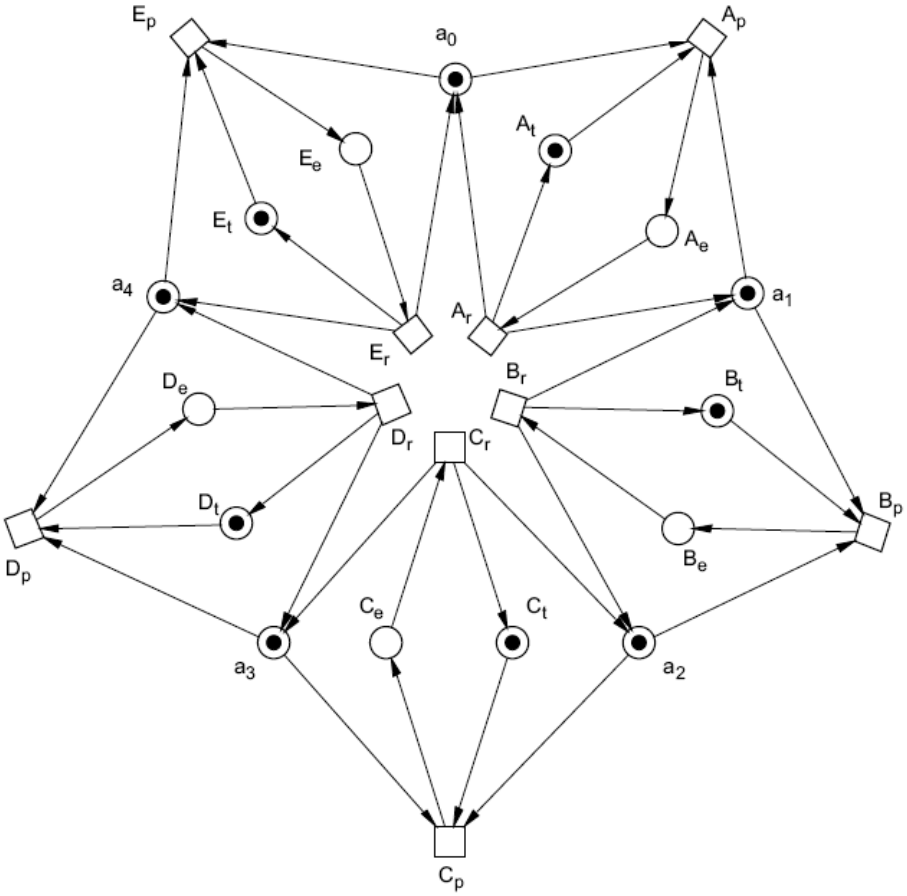
- The n tokens in p1 represent n processes that may want to read or write a shared memory represented by p3

Quando il writer va in CS prende tutti i token disponibili in P3 (che devono essere tutti i k, affinchè ci sia Mutua esclusione, cioè lettori non in CS): inoltre, solo un write in CS (vengono rispettati tutti gli invarianti)



DINING PHILOSOPHERS

- Forks are represented by places *ai*
- Philosophers thinking by A*t*,B*t*,C*t*,D*t*,E*t* places and eating by A*e*,B*e*,C*e*,D*e*,E*e* places

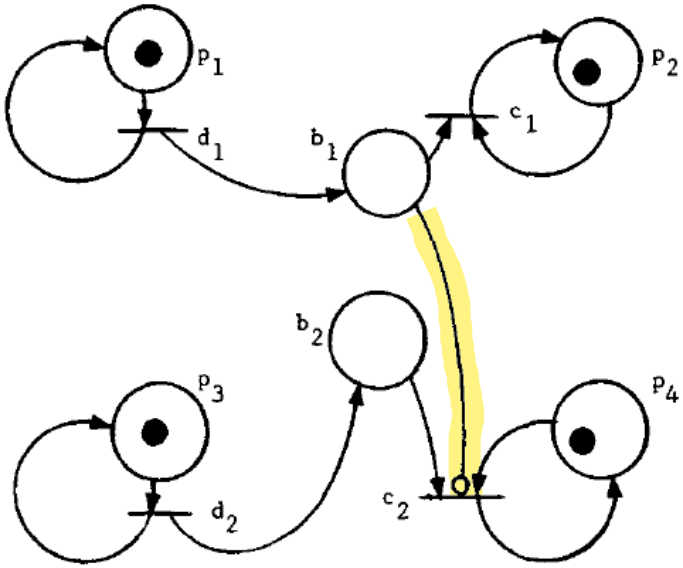


EXTENDED PETRI NETS WITH INHIBITOR ARCS

Una transizione collegata a un posto tramite un arco inibitore può scattare solo se quel posto è vuoto (zero token).

- Zero-testing extension
  - extending the basic PN with the possibility of firing a transition only if a certain place has zero tokens
    - *inhibitor arc* represented by an arc with a small circle at the end

Il limite delle Reti di Petri standard è che non possono "contare fino a zero" in modo semplice. Non esiste un meccanismo nativo per dire: "Fai scattare A solo se la coda B è vuota". Con l'arco inibitore, puoi implementare la logica "if then else":
- IF (Posto P è vuoto) THEN scatta la transizione T1.
- ELSE (Se c'è almeno un gettone) scatta la transizione T2.
Questo meccanismo permette di gestire le priorità. Ad esempio, in un sistema Lettori-Scrittori, puoi usare un arco inibitore per impedire a un Lettore di iniziare se c'è uno Scrittore in attesa, dando priorità assoluta alla scrittura.



  - PN+inhibitor arc = Turing-equivalent

Significa che hanno la stessa potenza di calcolo di qualsiasi linguaggio di programmazione moderno. Puoi simulare un intero computer usando solo piazze, transizioni e archi inibitori.

    - expressiveness and undecidability problems

Più il modello è espressivo (capace di descrivere sistemi complessi), meno è analizzabile matematicamente in modo automatico.

FORMAL DESCRIPTION OF PETRI NETS

- PN can be formally described ~~so as to enable~~ in modo da consentire a rigorous analysis of properties and problems of the system modelled
  - **structural** properties
    - independent from the initial marking

    Queste proprietà dipendono solo da come sono collegati frecce, piazze e transizioni, a prescindere dalla quantità dei token.
  - **behavioural** properties
    - dependent on the marking

    Se cambi il numero di token di partenza, queste proprietà possono cambiare drasticamente (perchè dipendono dai marking).
- Mapping correctness of the systems on to structural / behavioural properties of the nets
  - safety and liveness properties

Se stai progettando un sistema critico (es. il software di un'auto o di un dispositivo medico):
- Usi le proprietà strutturali per assicurarti che il design di base sia solido.
- Usi le proprietà comportamentali per testare scenari specifici (es. "Cosa succede se arrivano 1000 richieste contemporaneamente?").

Mappando la correttezza del software sulle proprietà della rete, puoi ottenere una certificazione formale del tuo sistema, riducendo la necessità di test empirici basati solo sulla fortuna.

PETRI NET STRUCTURE

- The structure of a Petri Net can be formally described as a tuple

$$C = (P,T,I,O)$$

  - P is a set of places
  - T is a set of transitions
  - input function I defines the set of *input* places for each transition t_j
  - output function O defines the set of *output* places for each transition t_j

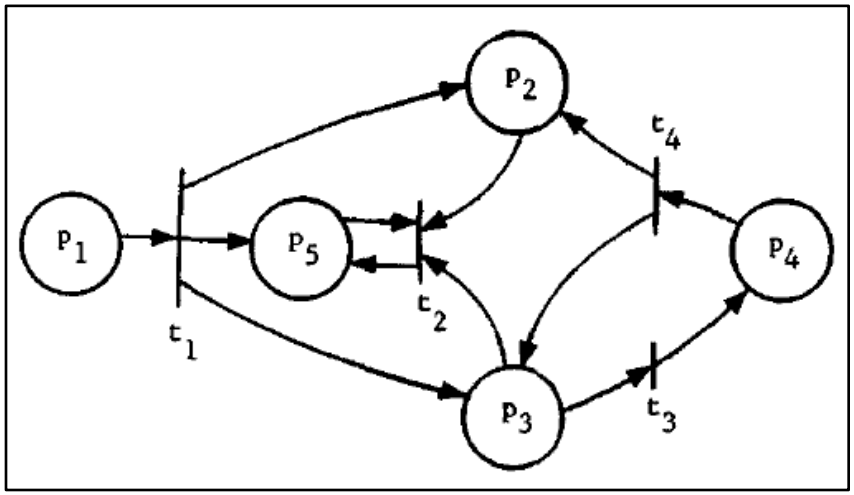
EXAMPLE

```
P = { p1, p2, p3, p4, p5 }
T = { t1, t2, t3, t4 }

I(t1) = {p1}
I(t2) = {p2,p3,p5}
I(t3) = {p3}
I(t4) = {p4}

O(t1) = {p2,p3,p5}
O(t2) = {p5}
O(t3) = {p4}
O(t4) = {p2,p3}
```

Corresponding Petri-Net graph:



MARKING

- A **marking** is an assignment of tokens to the places of the net
- Can be formally represented either as
  - a vector of N elements, one for each place, representing the number of tokens for each place
  - a function $\mu : P \longrightarrow N$

    - $\mu(p_i)$ is the number of tokens in the place p_i

Set di Marcature

Petri Net Structure

- A marked Petri Net is represented by 5-tuple: $M = (P, T, I, O, \mu)$

EXECUTION RULE SEMANTICS

- The state of a Petri Net is defined by is marking
  - firing of a transition represents a change in a the state of the net.
- Next-State partial function $\delta(\mu, t_j)$
  - the function is undefined if the transition is not enabled in the marking (cioè non può essere eseguita la transizione)
  - if t_j is enabled $\mu' = \delta(\mu, t_j)$ is the marking that results from removing tokens from the input of t_j and adding tokens to the output of t_j
- Given a PN and an initial marking, we can execute the PN by successive transition firings
  - firing a transition t_j in the initial marking produces a new marking $\mu^1 = \delta(\mu^0, t_j)$
  - in this new marking we can fire any new enabled transition, say t_k, resulting in a new marking $\mu^2 = \delta(\mu^1, t_k)$
  - this can continue as long as there is at least one enabled transition in each marking
  - if we reach a marking in which no transition is enabled, then no transition can fire and the PN must stop
- Non determinism
  - multiple sequence of markings $(\mu^0, \mu^1, \mu^2, \ldots)$ and related transitions $(t^0_j, t^1_j, t^2_j, , \ldots)$ can result by executing a PN

REACHABILITY SET

- **Immediately reachable** marking
  - a marking μ' is *immediately reachable* from μ if we can fire some enabled transition in μ resulting in μ'
- **Reachable marking**
  - a marking μ' is *reachable* from μ if it is immediately reachable from μ or is reachable from any marking μ'' which is immediately reachable from μ
- **Reachability set** of a PN
  - set of all states into which the PN can enter by any possible execution

MATRIX REPRESENTATION

- The components of a Petri net can be conveniently represented as matrices
  - *input* matrix D−, *output* matrix D+, *incident* matrix D
    - (m x n) matrix: m (rows) are transitions, n (columns) are places
    - Input matrix D−
      - element D−[i,j] = weight of arc connecting p_j with t_i
    - Output matrix D+
      - element D+[i,j] = weight of arc connecting t_i with p_j
    - Incident matrix D = D+ − D−

      Rappresenta la variazione netta della differenza tra token in input ed output.
  - *transition* matrix T

    È un vettore riga 1xm (spesso chiamato vettore di controllo). Contiene 0 (transizioni che non è possibile far scattare) e 1 nella posizione della transizione che è possibile far scattare.
    - (1 x m) matrix representing the firing of the Petri net, i.e. what transitions can be fire
  - *current marking* matrix M

    È un vettore riga 1xn che elenca il numero di gettoni correnti in ogni piazza.
    - (1 x n) matrix representing the current number of tokens in places
- To compute the evolution of a Petri Net: M' = T∗D + M

ANALYSIS OF PN MODELS

Una proprietà di Safety afferma che "qualcosa di male non accade mai". Nella Boundedness, se dimostri che una piazza ha al massimo k gettoni, stai garantendo che non accadrà mai che il buffer esploda. Nella Mutua Esclusione: Il "male" è avere due processi in una sezione critica. Se il posto è Safe (max 1 gettone), garantisci che non accadrà mai l'accesso simultaneo. Nella Conservazione: Il "male" è la perdita o la creazione magica di risorse. Garantisci che il numero totale di risorse non devierà mai dal valore iniziale.

- **Safe** nets

  Una rete si definisce Safe (o sicura) se in nessun posto possono esserci contemporaneamente più di un gettone (k=1).

  - Petri nets in which no more than one token can ever be in ~~any~~ nessun place of the net at the same time.
  - Justification based on the original definition of events and conditions
    - a condition is represented by a place
    - the fact that the condition holds is indicated by a token in the place
    - so a token should be either present or not: more than 2 token is pointless
- **Bounded net** or **k-bounded net** (*boundness*)

  La Boundedness è la generalizzazione della Safety. Una rete è k-bounded se il numero di gettoni in ogni posto non supera mai un valore massimo k.

  - Nets in which the number of tokens in any place is bounded by k
  - safe nets are 1-bounded net
  - Boundedness is a very important practical property
- **Conservative** net

  Una rete è Conservativa se il numero totale di gettoni all'interno del sistema non cambia mai, indipendentemente dalle transizioni che scattano.

  - PN is conservative if the number of tokens in the net is conserved
  - think for instance of using tokens to represent resources..

LIVENESS

Una transizione si dice morta in una certa marcatura se non esiste alcuna sequenza di scatti, per quanto lunga, che possa portarla a essere abilitata.
- Cosa significa: Quella parte del software o del sistema è diventata "codice morto". Non potrà mai più essere eseguita.
- Legame con il Deadlock: Se tutte le transizioni di una rete diventano morte, l'intero sistema è in deadlock totale. Nessun movimento è più possibile.

- Based on **transition analysis**
  - **dead transition** in a marking
    - if there is no sequence of transition firings that can enable it
    - related to *deadlock* situations
  - **potentially fireable**
    - if there exists some sequence that enables it
    - related to starvation situation
  - **live transition**
    - if it is potentially fireable in all reachable markings
- For liveness, it is important not only that a transition be fireable in a given marking, but *staying potentially fireable in all markings reachable from that marking*
  - if it is not true, then it is possible to reach a state in which the transition is dead => deadlock

attivazione

Una transizione è potenzialmente attivabile se esiste almeno una sequenza di scatti che la abilita partendo dalla marcatura attuale. Il rischio Starvation (Inedia): Una transizione può essere potenzialmente attivabile ma non scattare mai.

Questa è la condizione ideale per i sistemi che devono operare indefinitamente (come un sistema operativo o un server). Una transizione è viva se è potenzialmente attivabile in tutte le marcature raggiungibili. Cosa significa: Non importa cosa succeda o quali scelte vengano fatte, avrai sempre la possibilità di far scattare quella transizione in futuro. Non potrai mai "incastrarti" in un angolo della rete dove quella funzione diventa inutilizzabile. Resilienza: Un sistema composto solo da transizioni vive è un sistema che può sempre recuperare e tornare al suo ciclo di lavoro principale.

PETRI NET EXTENSIONS

- **Timed Nets**
  - introducing time delays associated with transitions and/or places
    - useful for performance evaluation and scheduling problems
  - deterministic timed nets
    - delays are deterministically given
  - **stochastic nets**
    - delays are probabilistically specified
- **High-level Nets**
  - associate some kind of symbolic / numerical information to tokens and some computational rules to transition consuming and producing tokens
    - Coloured Petri Nets, Predicate Transitions Nets
  - **Coloured Petri Nets**
    - assigning typed values (“a color”) to each token

PETRI NET TOOLS

- Many tools available for creating and analyzing Petri Nets
  - check http://www.informatik.uni-hamburg.de/TGI/PetriNets/
- Examples:
  - PIPE 2
    - Platform Independent Petri net Editor 2
    - Java-based, open-source: http://pipe2.sourceforge.net/
  - TINA
    - Time Petri Nets analyzer
    - https://projects.laas.fr/tina/index.php

NON ENTRIAMO NEI STATE CHART PERCHÉ LI ABBIAMO GIÀ VISTI

STATECHARTS

STATECHARTS

- Introduced by David Harel in 1987 for modelling *complex reactive systems*
  - now part of UML with the name of state diagrams
- Reactive systems
  - systems being, to a large extent, *event-driven*, continuously having to react to external and internal stimuli
    - examples include automobiles, communication networks, operating systems, avionic systems, man-machine interface of many ordinary software
  - contrast to *transformational* systems
    - input / output, data-processing systems
- Main objective
  - introducing a way of describing reactive behaviour that is clear and realistic, and at the same time formal and rigorous
    - to be simulated and analysed

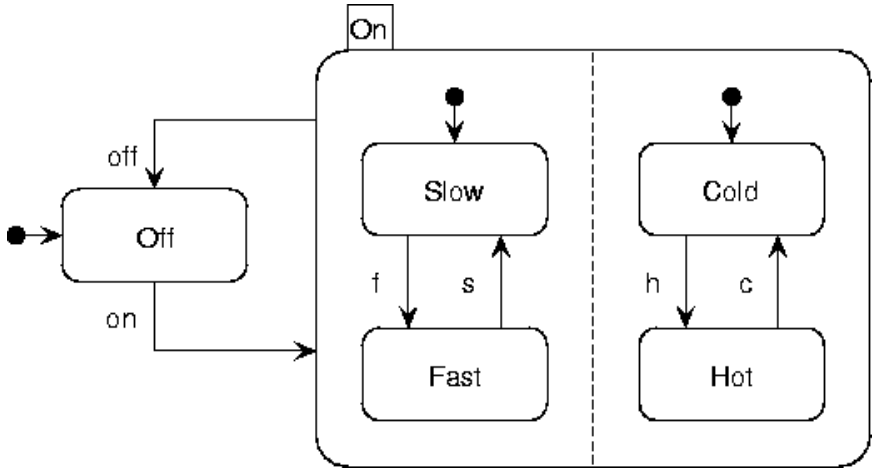
BEYOND BASIC STATE DIAGRAMS

- General agreement that **states** and **events** are a rather natural medium for describing the dynamic behaviour of a complex systems
  - state transition: "when event alfa occurs in state A, if condition C is true at that time, the system transfers to state B"
- But finite state machine and state transition diagrams don’t scale with complexity
  - unmanageable, exponentially growing multitude of states, all of which have to be arranged in a "flat" unstratified manner
  - lead to unstructured, unrealistic and chaotic state diagram
- To be useful a state/event approach must be *modular*, *hierarchical* and *well-structured*
  - it must solve the exponential blow-up problem, by somehow relaxing the requirement that all combinations of states have to be represented explicitly
- Statecharts proposal
  - extension of conventional state diagrams by mechanisms to enhance the descriptive power

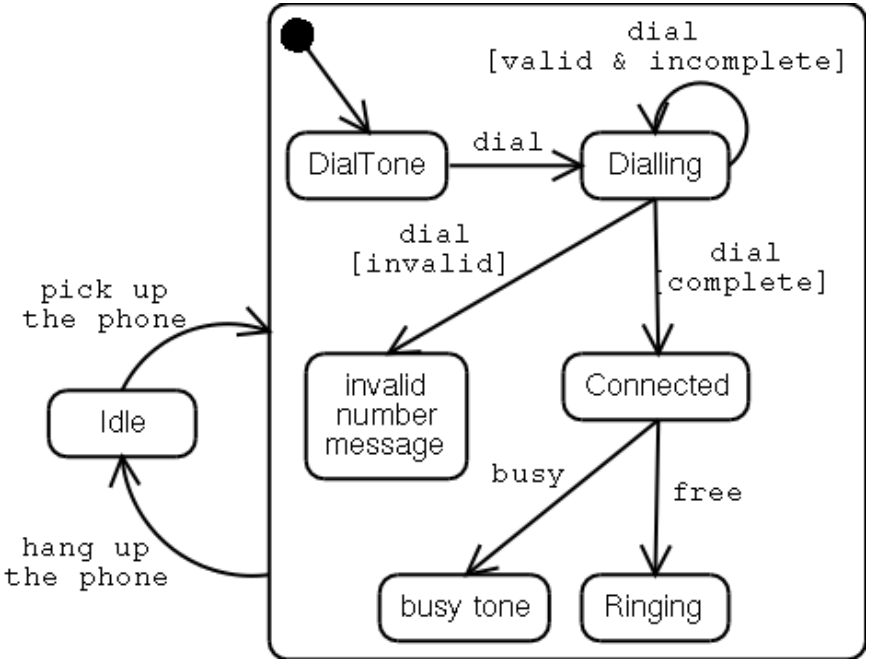
STATECHARTS FORMALISM

- Visual formalism to describe states and transitions in a modular fashion
  - **hierarchy**
    - clustering
    - refinement
    - promoting 'zoom' capabilities for moving easily back and forth between levels of abstractions
  - **orthogonality**
    - independence / concurrency of substates
    - synchronisation among substates

A TASTE



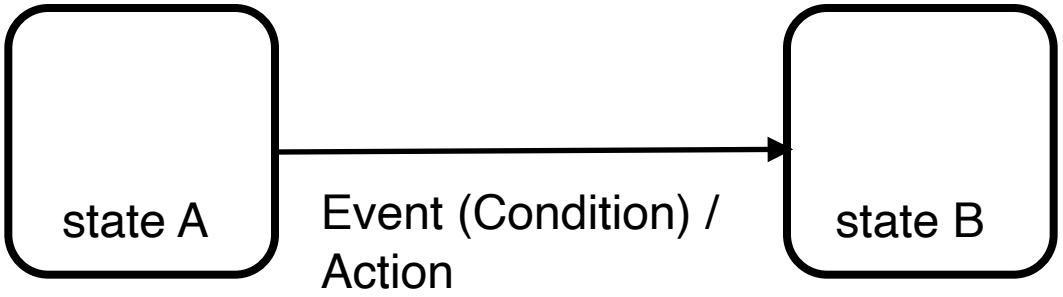
A fan



A phone call

STATE AND EVENTS IN STATECHARTS

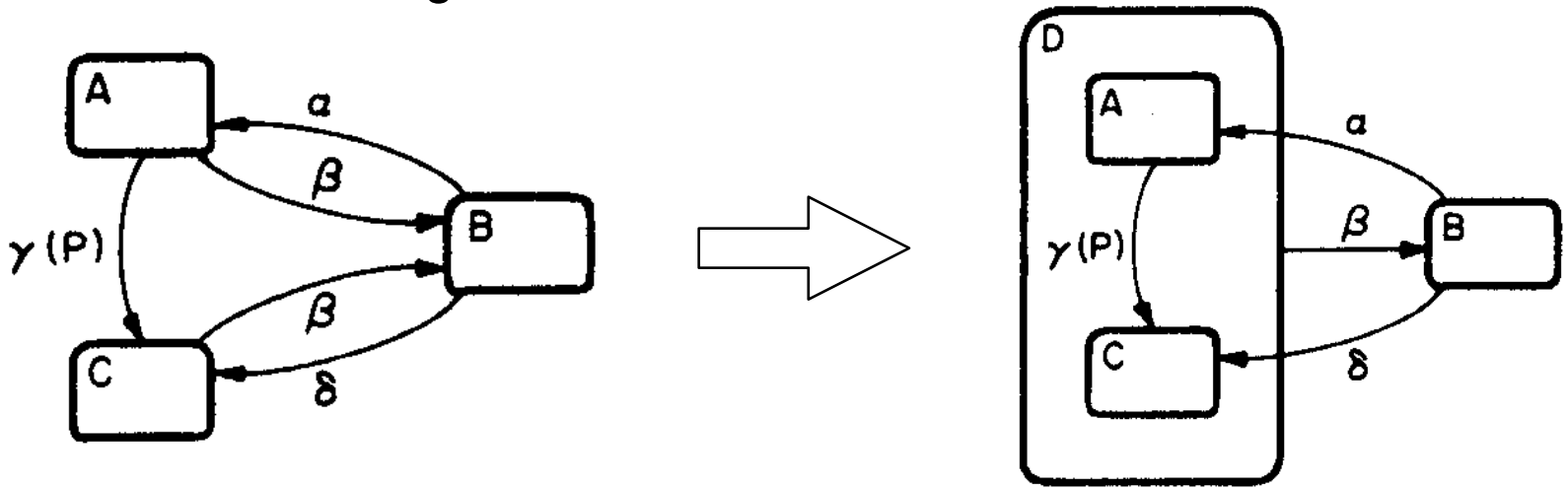
- States and events
  - **boxes** (rounded rectangle) denotate **states**
  - **arrows** labelled with **event**
    - optionally with a parenthesised **conditions** and an **action** (described later on)



- Different state levels (= hierarchy support)
  - encapsulation express hierarchies
  - arrows can originate and terminate at any level

HIERARCHY: CLUSTERING

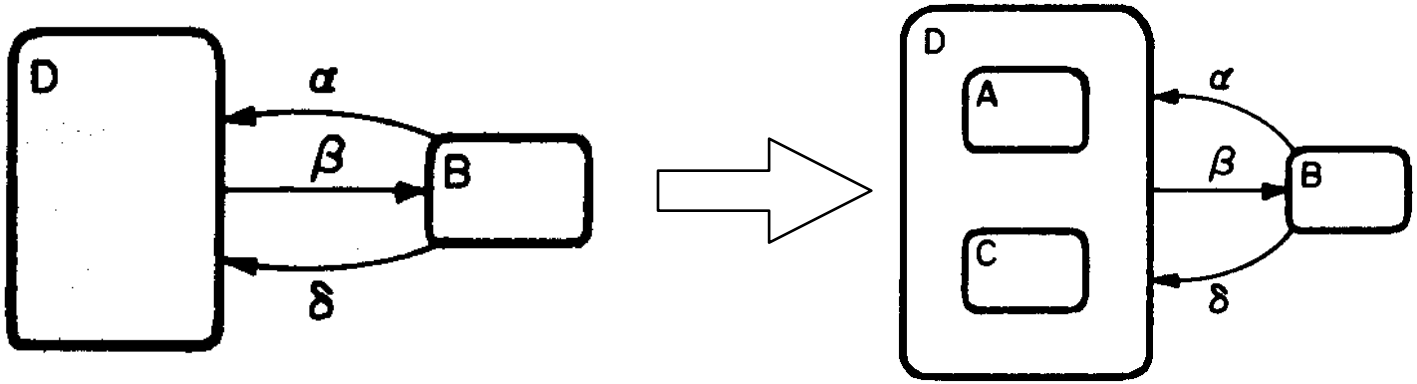
- XOR Decomposition
  - economising arrows



- since beta takes the system to B from either A or C, we can cluster the latter into a new super-state D and replace the two arrows by one
  - the semantics of D is a XOR of A and C: to be in state D one must be either in A or in C, and not in both.
  - D is an *abstraction* of A and C
    - capturing common properties
- bottom-up approach

REFINEMENT

- We can proceed on the opposite direction, refining states:

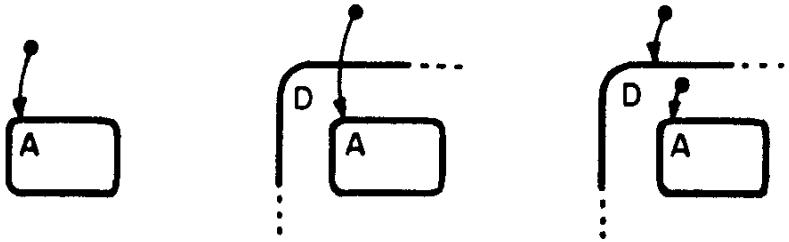


  - in this case the incoming alfa and beta arrows are underspecified
- top-down approach

- Zooming in and out support
  - zooming-in
    - by looking inside a state
  - zooming-out
    - abstracting from the inside of a state

DEFAULT STATES

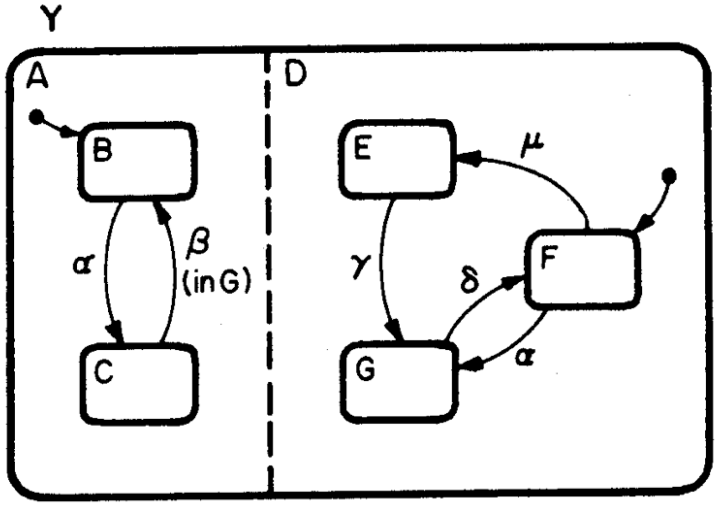
- Special arrow to explicitly represent the default entering state
  - at any level



- Take into the account the history
  - entering the state most recently visited
  - H default-state arrows

HIERARCHY 2: ORTHOGONALITY

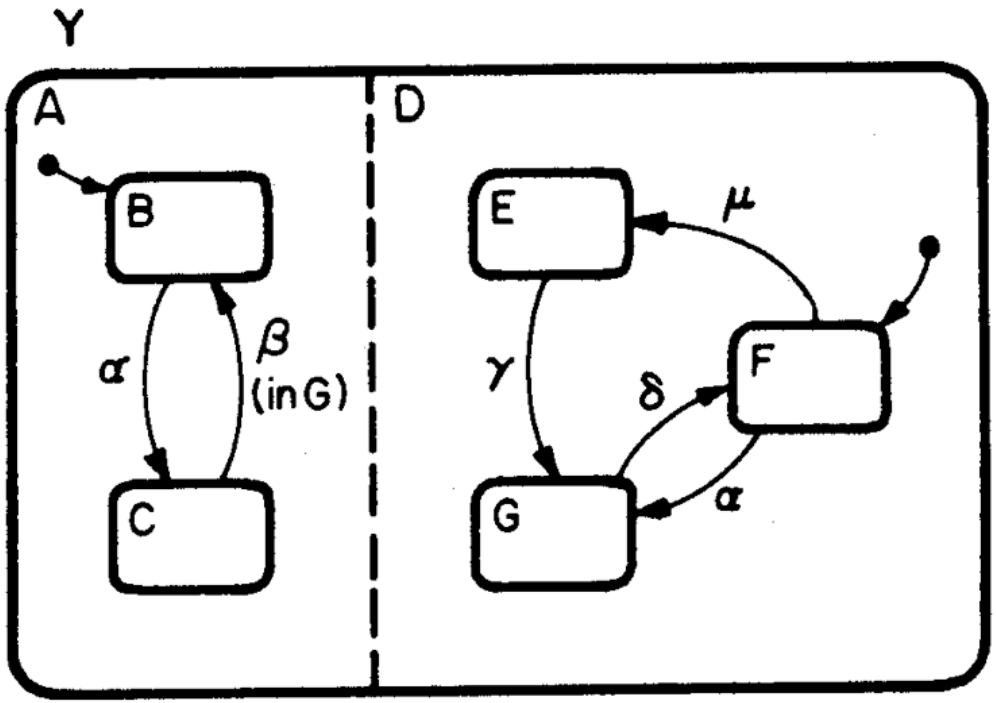
- AND decomposition
  - capturing the property that, being in a state, the system must be in all of its AND components
  - the notation used in statecharts is the physical splitting of a box into component using dashed lines



  - state Y consists of AND components A and D
    - with the property that being in Y entails being in some combination of B or C with E, F or G.
    - Y is the orthogonal product of A and D
  - Independency and / or Concurrency

INDEPENDENCE

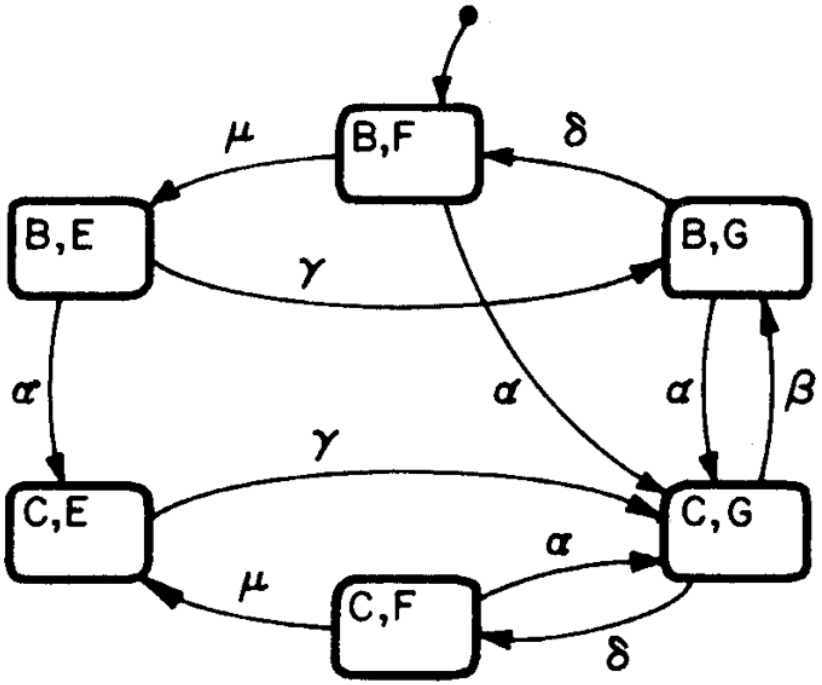
- On the other hand, μ occurs at (B,F) it affects only the D component, resulting in (B,E)



- This illustrates a certain kind of independence
  - the transition is the same whether the system is in B or C in its A components

AND-FREE EQUIVALENT

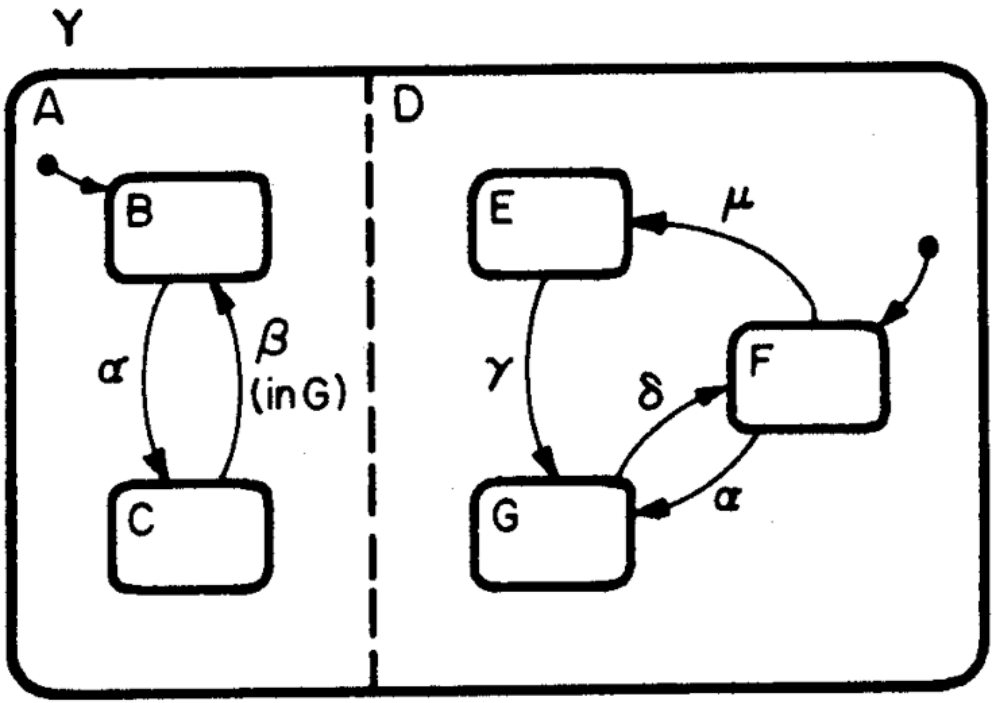
- The AND-Free equivalent diagram has the product of the states



- That is: if we have two components with 1000 states, we have one million of states in the product
  - if we have 3 components: 10^9 states..

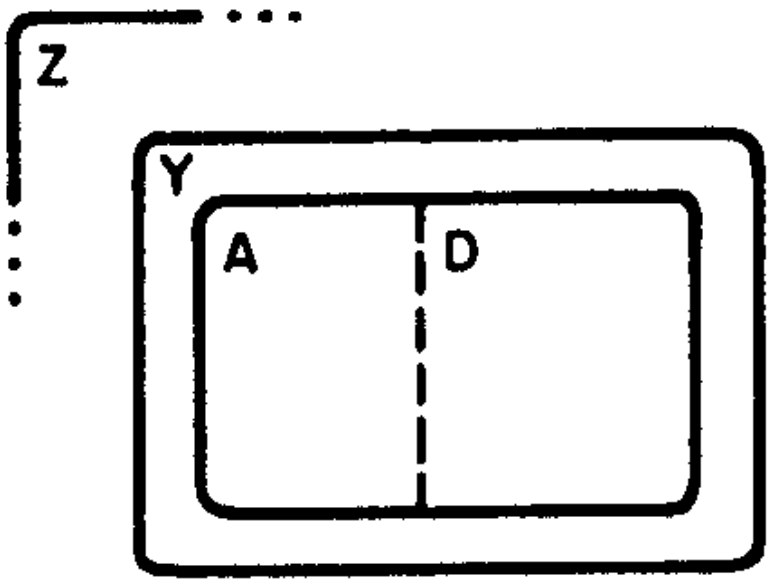
SYNCHRONISATION

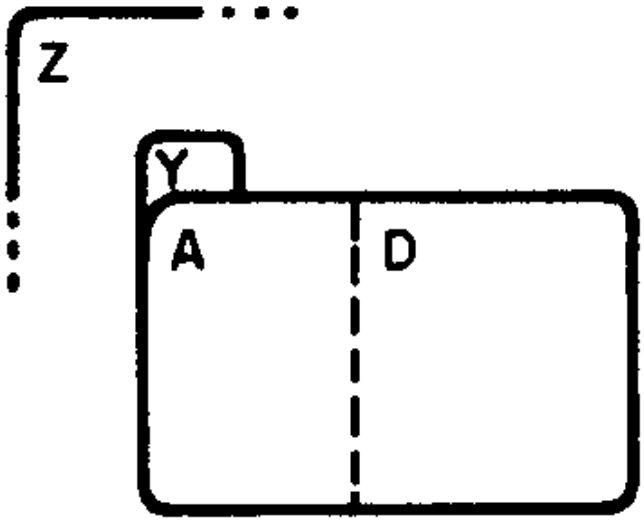
- In the example, if an event alfa occurs, it transfers B to C and F to G simultaneously, resulting in a new combined state (C,G)



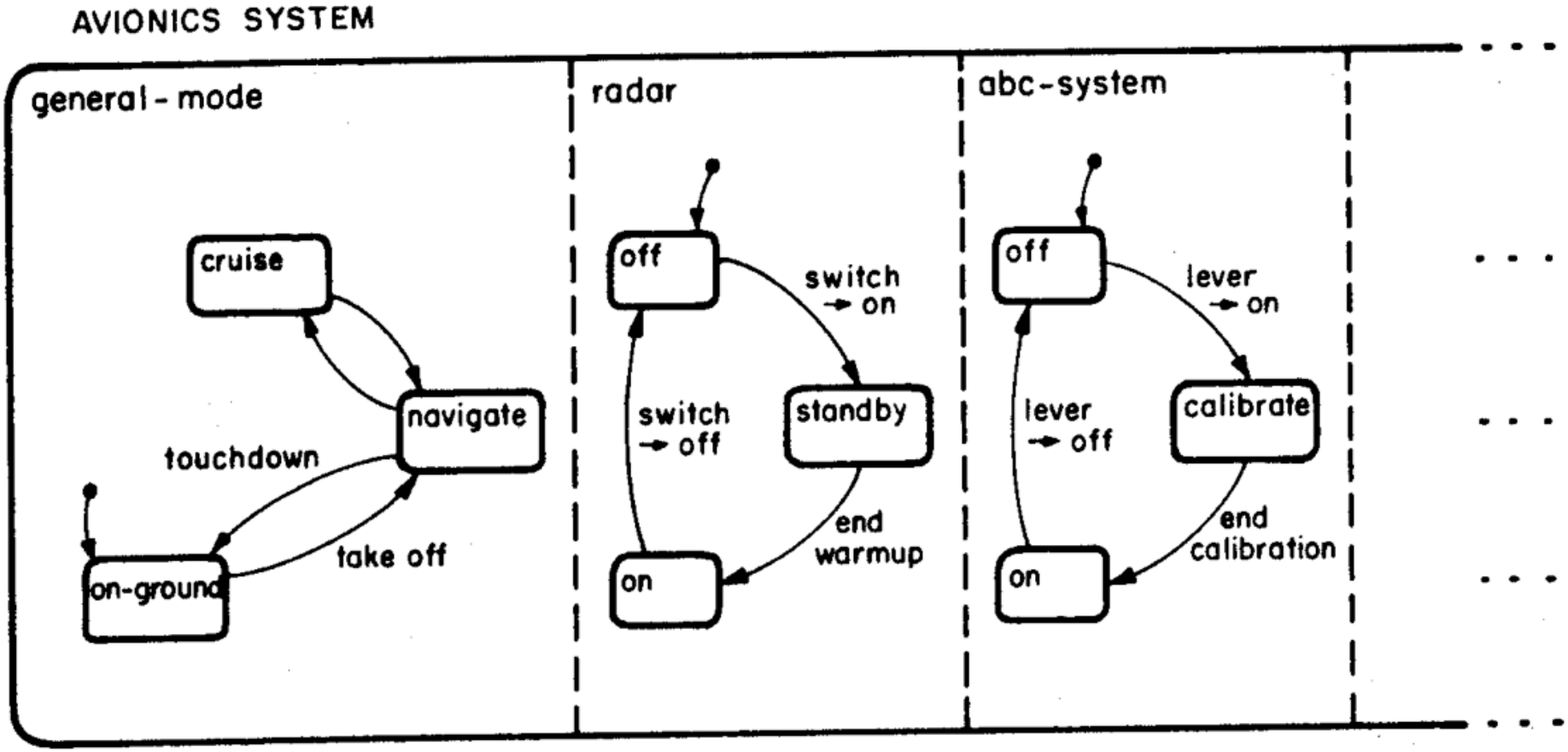
- This illustrates a certain kind of synchronisation
  - a single event causing simultaneous happenings

NOTATIONS FOR AND-DECOMPOSITION



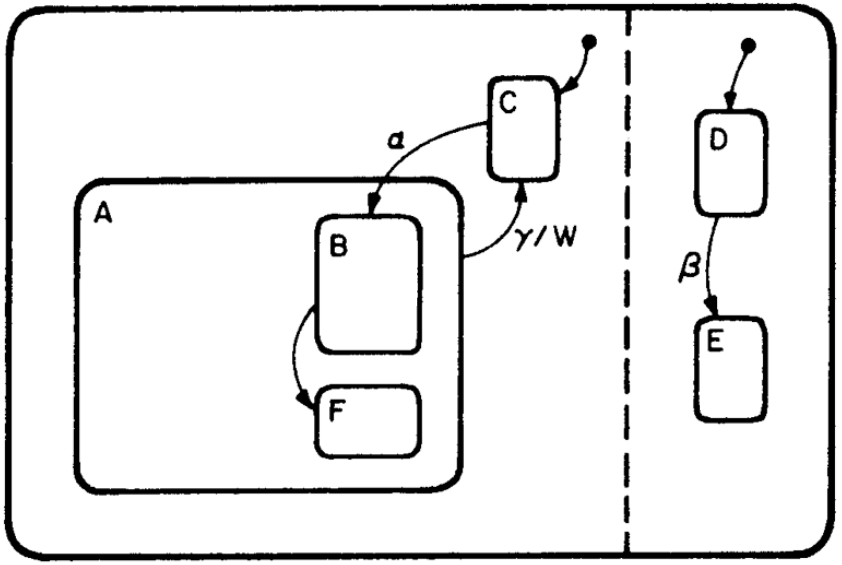


AN EXAMPLE


AVIONICS SYSTEM
general - mode
cruise
navigate
touchdown
take off
on-ground
radar
off
switch → on
standby
switch → off
end warmup
on
abc-system
off
lever → on
calibrate
lever → off
end calibration
on

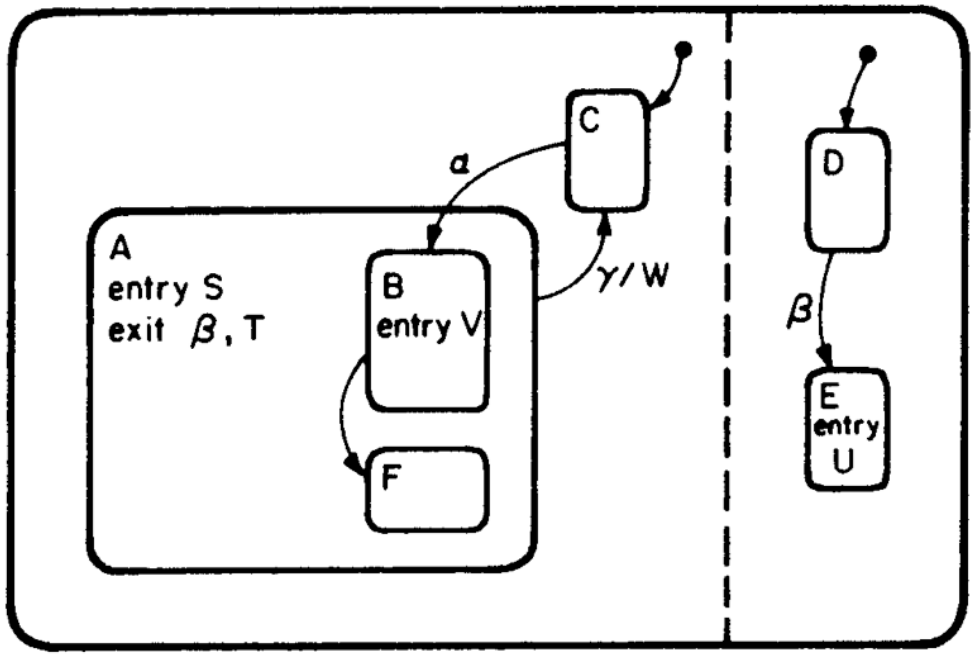
ACTIONS (1/2)

- Actions represents the ability of the statecharts to generate events and to change the value of conditions
  - influencing other components of the system
  - influencing the environment of the system
- Expressed by the notation ".../W" that can be attached to the label of the transition
  - W is an action carried out by the system
  - actions have instantaneous occurrences that take ideally 0 time.



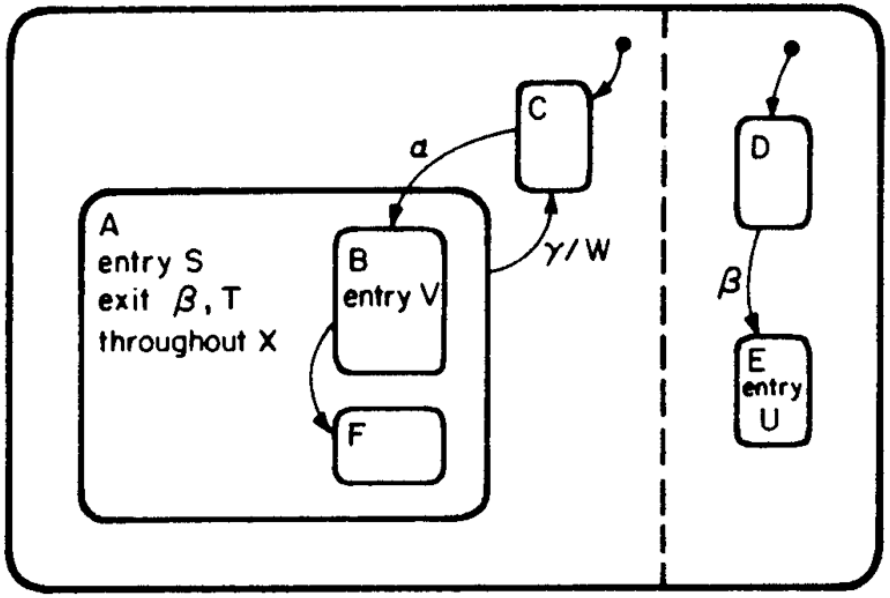
ACTIONS (2/2)

- Actions can be produced also when entering or when leaving a state
  - entry action (S, V, U in the example)
  - exit action (T in the example)



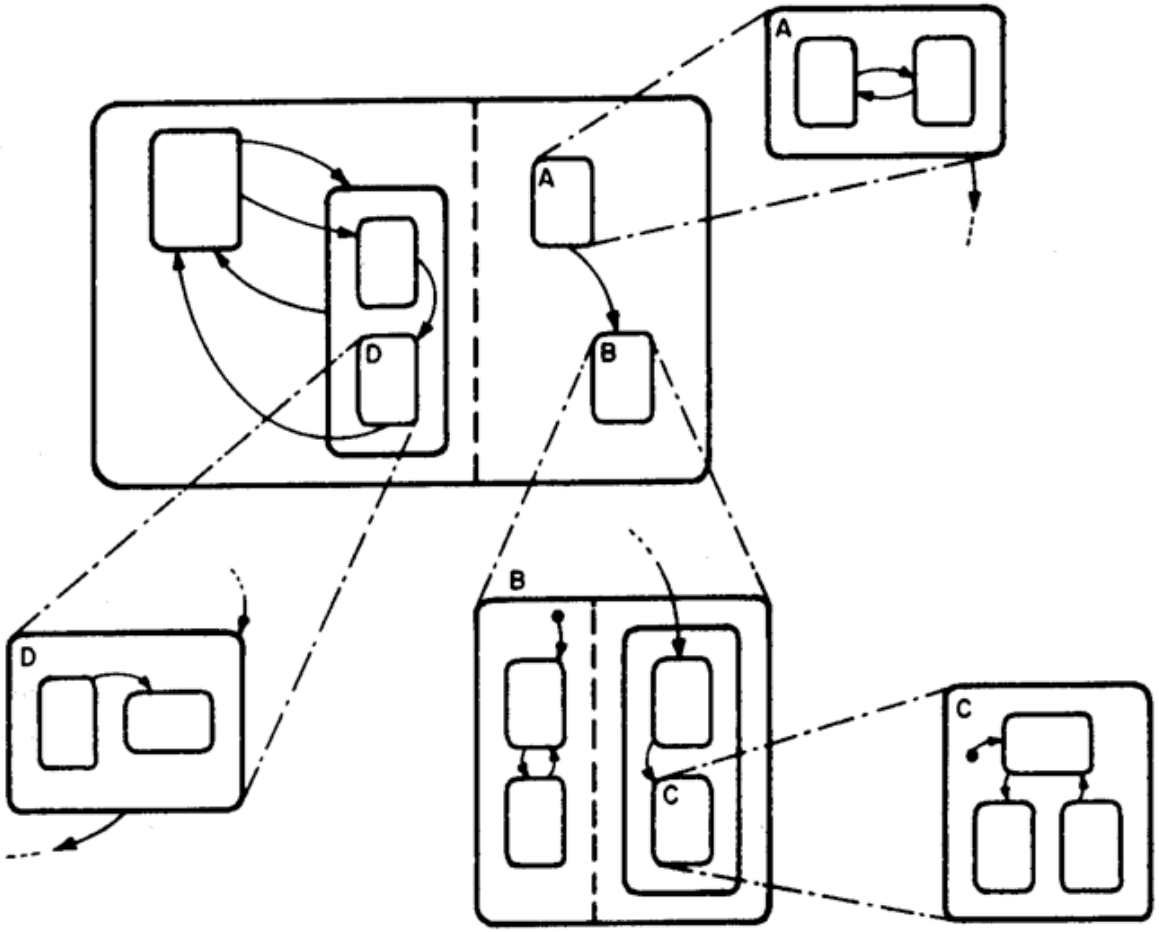
REPRESENTING ACTIVITIES

- Activities always take a nonzero amount of time (like beeping, displaying, executing lengthy computations..)
  - activities are durable
- So in statecharts activities are associated with states
  - exploiting entry and exit actions



FURTHER FEATURES: UNCLUSTERING

- Laying out parts outside the natural neighbourhood



THE STATEMATE TOOL

- Statemate is a comprehensive graphical modelling and simulation tool for the rapid development of complex embedded systems based on statecharts
  - using a combination of traditional graphical design notations combined with some of the Unified Modeling Language (UML) diagrams
- Statemate provides a direct and formal link between user requirements and software implementation by allowing the user to create a complete, executable specification
  - this specification may be executed, or graphically simulated, so the system engineer can explore what-if scenarios to determine if the behavior and the interactions between system elements are correct
  - these scenarios can be captured and included in Test Plans which are later run on the embedded system to ensure that what gets built meets what was specified.
  - this executable specification is also used to communicate with

NON ENTRIAMO NEI ACTIVITY DIAGRAMS PERCHÉ LI ABBIAMO GIÀ VISTI

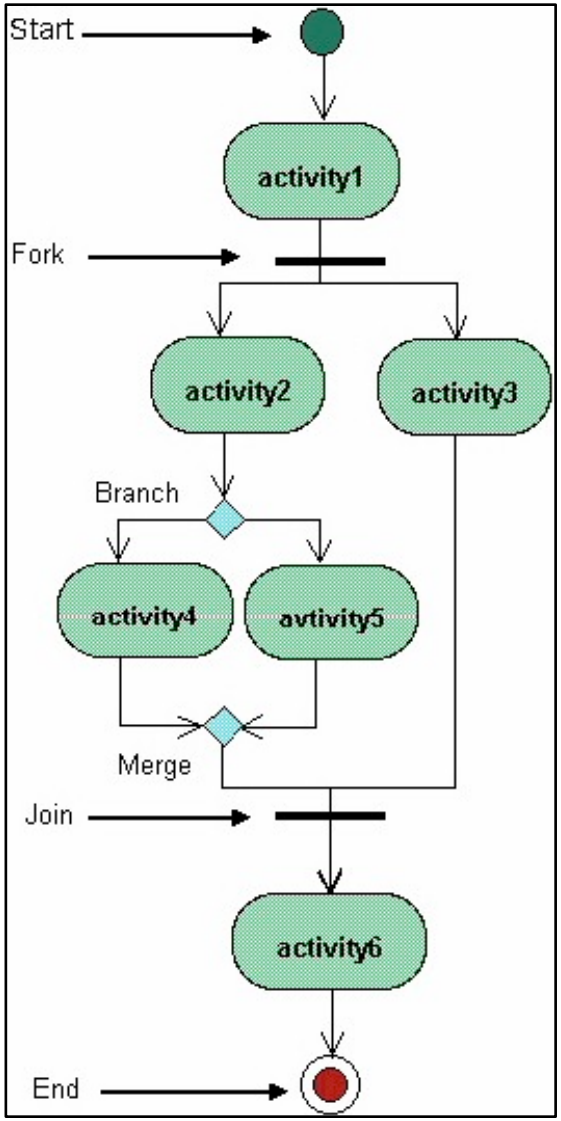
ACTIVITY DIAGRAMS

ACTIVITY DIAGRAMS

- Activity diagrams are one of the diagrams adopted in UML to represent the business and operational *workflows* of software system
  - an activity diagram is a dynamic diagram that shows the activity and the events that causes the object to be in the particular state
- Activity diagrams vs. state diagrams
  - a state diagram shows the different states an object is in during the lifecycle of its existence in the system, and the transitions in the states of the objects
    - these transitions depict the activities causing these transitions, shown by arrows
  - an activity diagram talks more about these transitions and activities causing the changes in the object states

ACTIVITY DIAGRAM OVERVIEW (*)

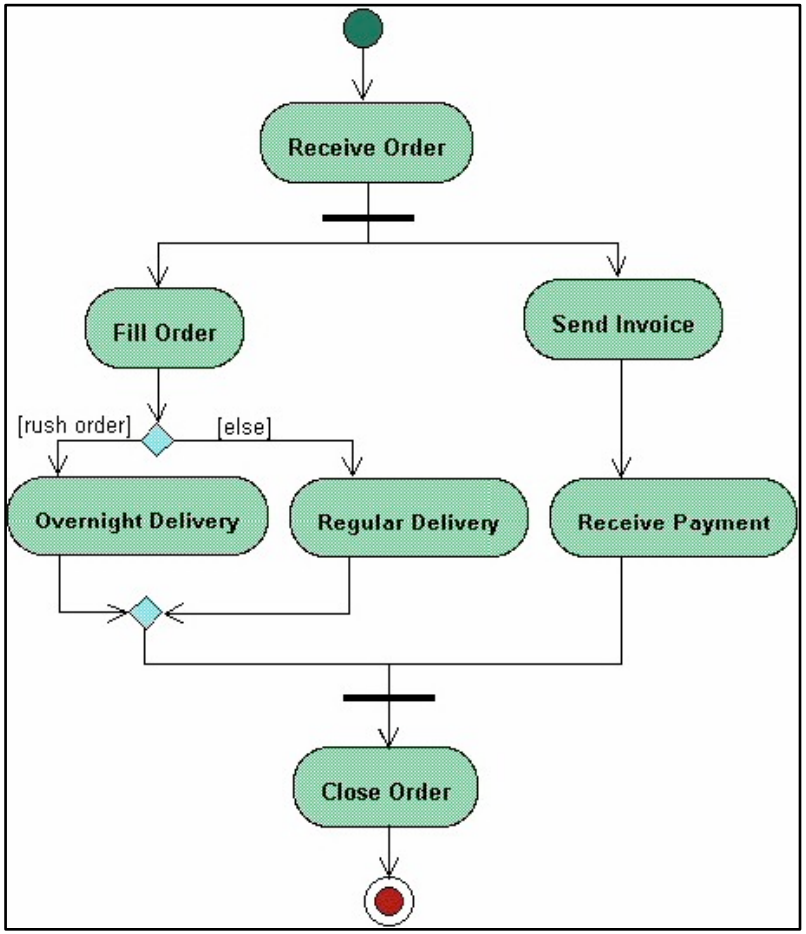
- Showing the flow of activities through the system
  - diagrams are read from top to bottom and have branches and forks to describe conditions and parallel activities. A fork is used when multiple activities are occurring at the same time.
- The diagram below shows a fork after activity1
  - this indicates that both activity2 and activity3 are occurring at the same time. After activity2 there is a branch. The branch describes what activities will take place based on a set of conditions. All branches at some point are followed by a merge to indicate the end of the conditional behavior started by that branch. After the merge all of the parallel activities must be combined by a join before transitioning into the final activity state.



(*) taken from http://atlas.kennesaw.edu/~dbraun/csis4650/A&D/UML_tutorial/activity.htm

PROCESSING ORDER EXAMPLE (*)

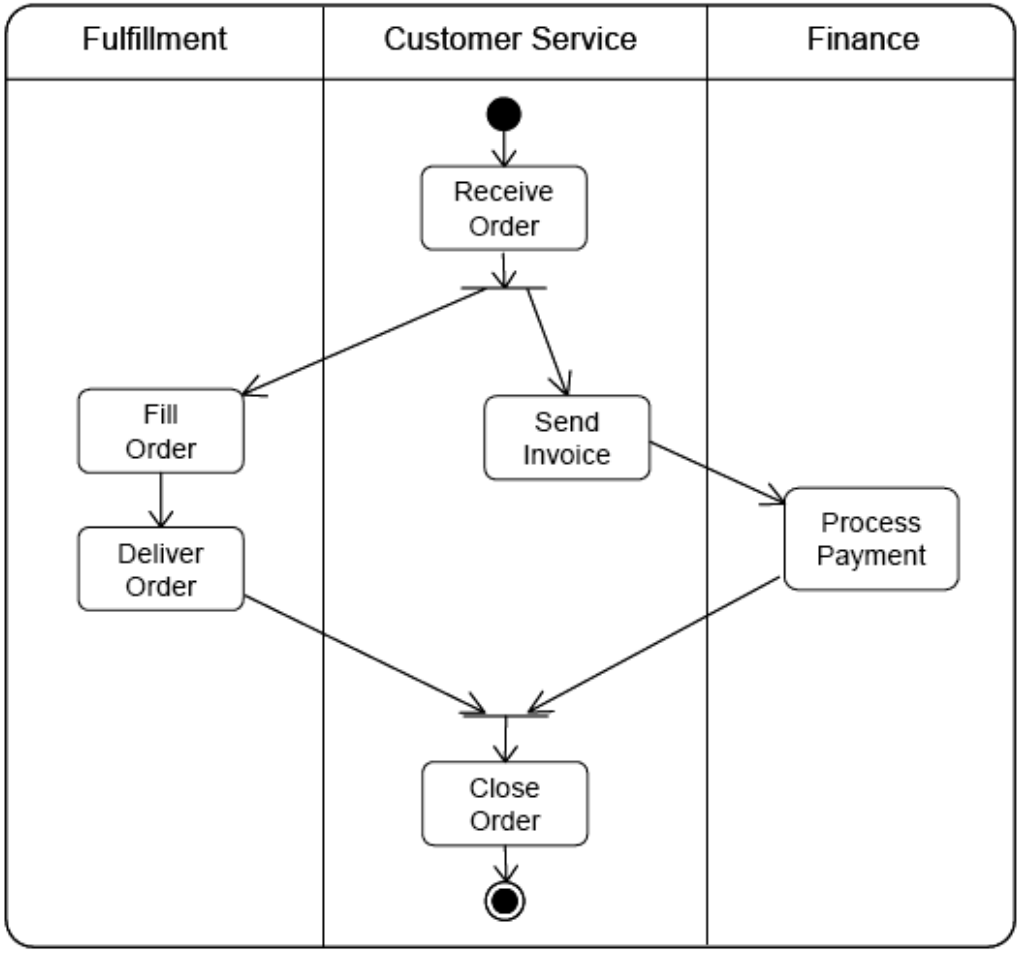
- The diagram shows the flow of actions in the system's workflow
  - once the order is received the activities split into two parallel sets of activities
  - one side fills and sends the order while the other handles the billing
  - on the Fill Order side, the method of delivery is decided conditionally.
  - depending on the condition either the Overnight Delivery activity or the Regular Delivery activity is performed.
  - finally the parallel activities combine to close the order



(*) taken from http://atlas.kennesaw.edu/~dbraun/csis4650/A&D/UML_tutorial/activity.htm

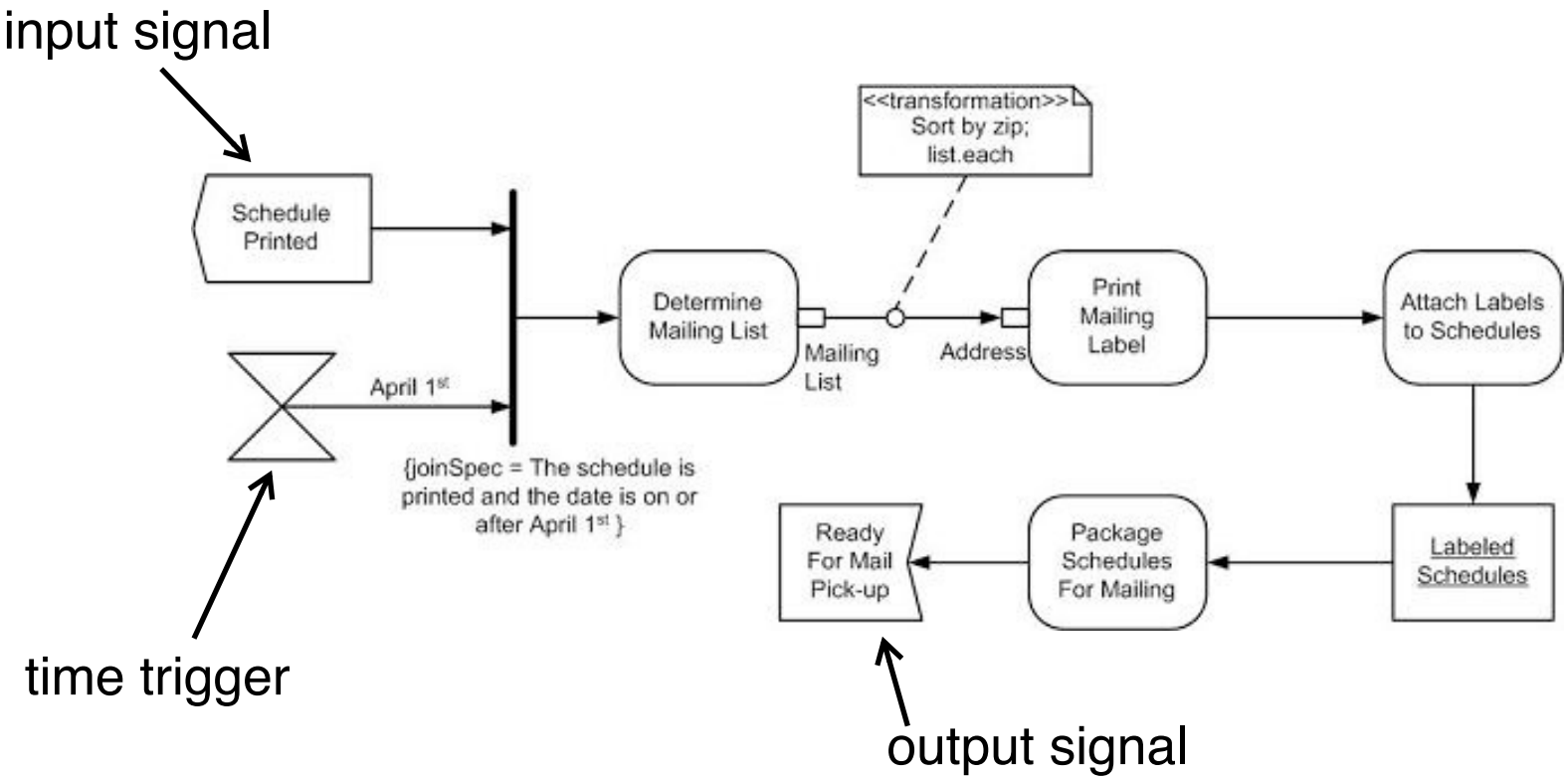
SWIMLANES

- A *swimlane* is a way to group activities performed by the same actor on an activity diagram or to group activities in a single thread



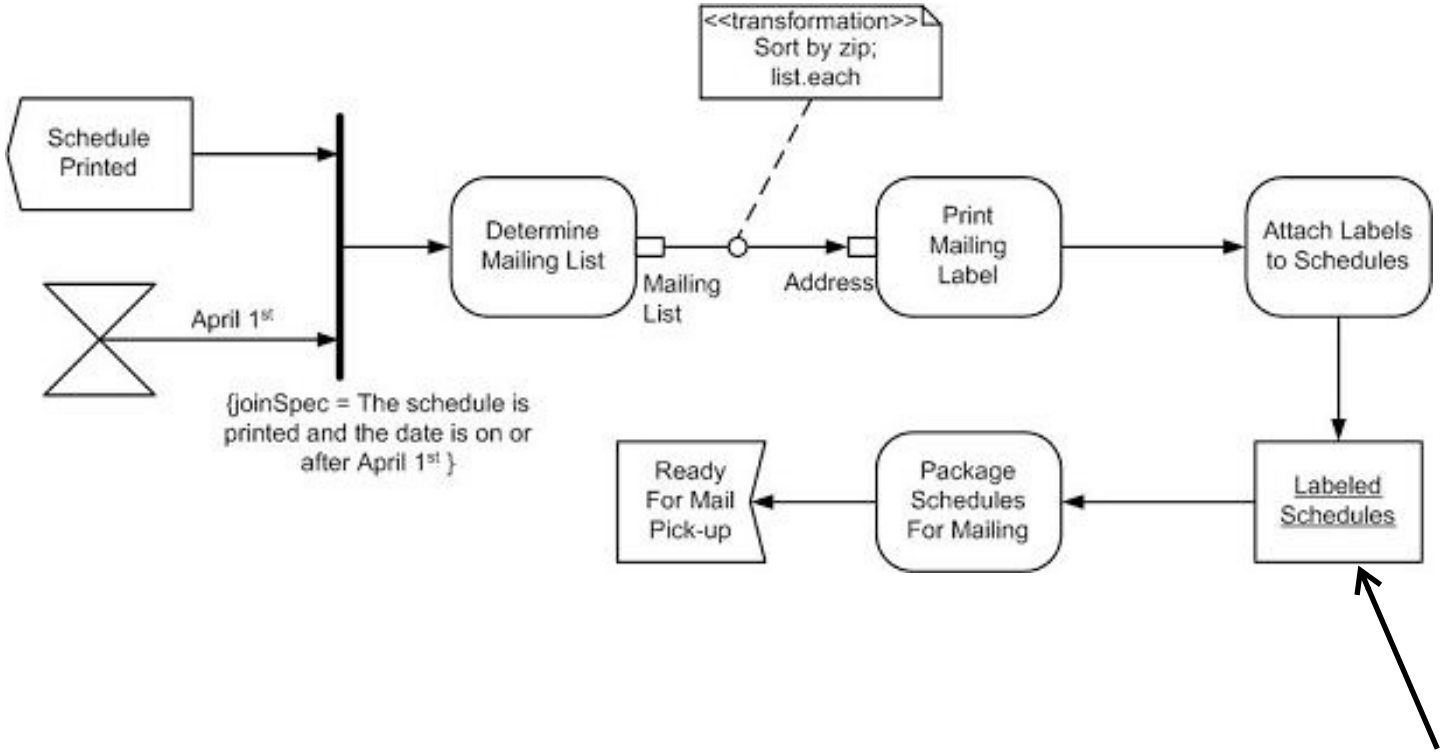
SIGNALS

- *Signal* Activities
  - activities that send or receive messages (output / input signals)
- Triggers
  - temporal signals



PASSING OBJECTS

- Specifying objects passed between activities



object passed between activities

BIBLIOGRAPHY

- David Harel, “Statecharts: A Visual Formalism for Complex Systems”. *Science of Computer Programming*, 8 (1987)
- David Harel, “On Visual Formalisms”, *CACM*, Vol. 31, No. 5, 1988
- James Peterson, "Petri Nets”, *ACM Computing Surveys (CSUR)*, Volume 9, Issue 3, Sept. 1977
- Tadao Murata, “Petri Nets: Properties, Analysis and Applications”, *Proceedings of the IEEE*, Vol. 77, No:4, April 1989